CEH v13 AI (312-50) Study Guide

Module 20: Cryptography

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
4. Tool Encyclopedia
5. Command Sheet (I have no lab, so teach me to read commands)
6. Ports, Protocols, Standards and Acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
8. Exam Cheat Sheet
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report

Source: 26 pages of screenshots from the official Module 20 theory videos (slides 2 to 55 plus the module summary). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry in the source to read with full confidence. Page numbers in this guide ("page 9") are the PDF page numbers of your notes. Slide numbers ("slide 20") are the small numbers printed on the slides.

1. Module at a Glance

Module: 20 - Cryptography (EC-Council CEH v13 AI). This is the last of the 20 modules.

What this module is (5 lines):

1. It explains what cryptography is, its four goals, and the difference between symmetric, asymmetric, and hash-based protection.
2. It catalogs the algorithms: symmetric (DES to ChaCha20), asymmetric (RSA, DSA, Diffie-Hellman, ECC, ElGamal), and hashes (MD2 to BLAKE3).
3. It covers where cryptography is applied: PKI and certificates, digital signatures, SSL/TLS, PGP/GPG, S/MIME email, disk encryption, hardware encryption, blockchain, and quantum cryptography.
4. It covers attacks: cryptanalysis methods, attack types, brute force (with a VeraCrypt demo), side-channel, DUHK, DROWN, rainbow tables, five blockchain attacks, and 13 quantum-era attacks.
5. It ends with countermeasures (12 general, 10 quantum) and key stretching (PBKDF2, bcrypt).

Why it matters: Every other module depends on cryptography. HTTPS, VPNs, Wi-Fi, passwords, code signing, and cloud storage all rely on it. A tester must know which algorithm is weak, which attack fits which weakness, and which fix is correct.

What the exam usually asks from it:

- Match an algorithm to its facts: key size, block size, block or stream, symmetric or asymmetric, hash output length.
- Pick the right key for the job (encrypt with the receiver's public key, sign with your own private key).
- Name the attack from a description (DROWN, DUHK, Finney, Eclipse, meet in the middle, rainbow table, side-channel, rubber hose).
- Identify the attack type from what the attacker knows (ciphertext-only, known-plaintext, chosen-plaintext, chosen-ciphertext).
- Name PKI parts, SSL/TLS handshake steps, and the right tool for a job (VeraCrypt, BitLocker, GPG, hashcat, CrypTool).
- Choose the best defense: key sizes, salting, key stretching, no hard-coded keys.

The 10 things to remember most:

1. Four goals: Confidentiality, Integrity, Authentication, Nonrepudiation.
2. Symmetric = one shared key, fast. Asymmetric = key pair, slow. Real systems (TLS, PGP) mix both (hybrid).
3. Encrypt with the receiver's PUBLIC key. Sign with the sender's PRIVATE key. Verify with the sender's PUBLIC key.
4. Key sizes: DES 56, 3DES 112/168, AES 128/192/256, RSA 2048 or more, hash 256 bits or more.
5. Hashes are one-way: MD5 128 bits, SHA-1 160 bits, SHA-2 224 to 512, SHA-3 uses a sponge. MD5 and SHA-1 are broken for collisions.
6. PKI parts: CA issues, RA verifies the applicant, VA answers validity checks (CRL and OCSP), certificates carry the public key.
7. TLS has two sub-protocols: Handshake (agree and exchange keys) and Record (protect the data).
8. PGP and GPG are hybrid and use the Web of Trust. S/MIME uses CA certificates.
9. Attack types by knowledge: ciphertext-only, known-plaintext, chosen-plaintext, adaptive chosen-plaintext, chosen-ciphertext.
10. Defend with strong keys, salts, key stretching (PBKDF2, bcrypt), no keys in source code, standard algorithms (never homemade).

Learning objectives of this module (page 1, slide 2):

#	Objective	Where covered
01	Cryptography concepts and encryption algorithms	3.1 to 3.12
02	Applications of cryptography	3.13 to 3.25
03	Cryptanalysis methods and cryptography attacks	3.26 to 3.43
04	Cryptography attack countermeasures	3.44 to 3.46

Module summary of the official notes (page 26): basic concepts and types; ciphers and algorithms; cryptography tools; PKI; email encryption protocols and tools; disk encryption and tools; cryptanalysis methods and code-breaking methodologies; cryptanalysis attacks and tools; and countermeasures. Each item maps to the sections in the table above.

2. Big Picture Map

Cryptography has three building blocks (symmetric, asymmetric, hashing). Applications combine them. Attacks target the algorithm, the key, the implementation, or the human. Defenses close each gap.

Diagram: building blocks, applications, attacks, defense

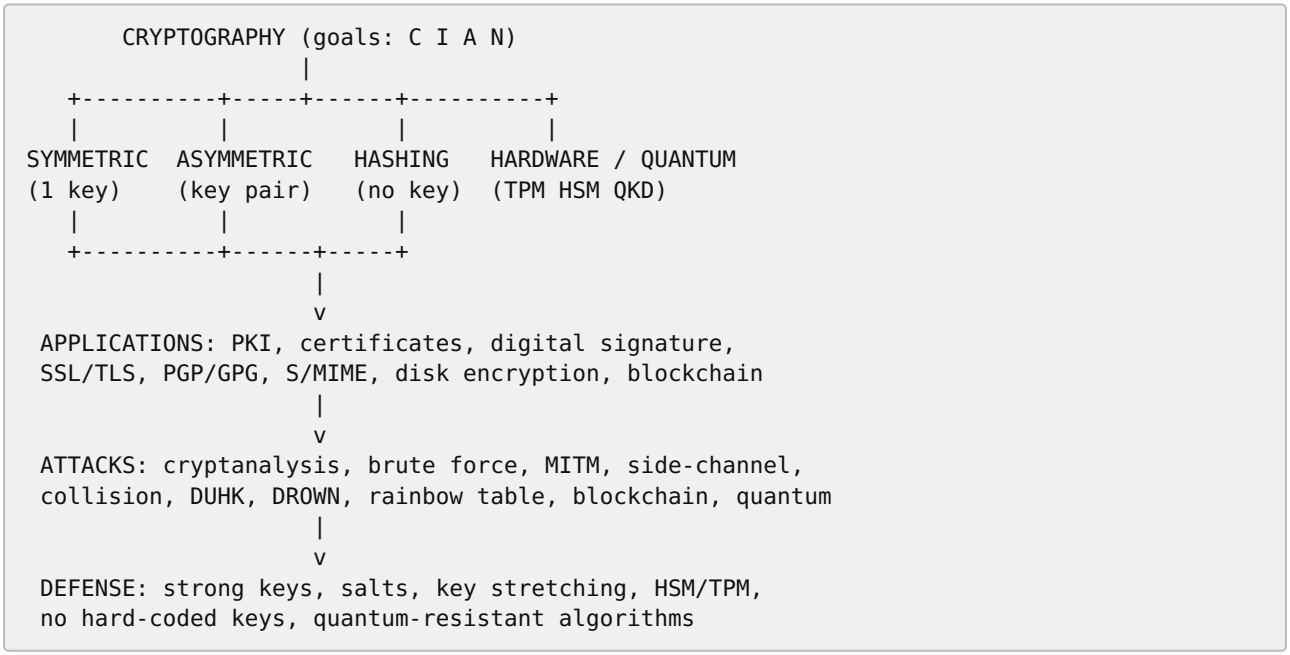


Table: how the topics connect to the C I A N goals

Goal	Main tool	Typical application	Typical attack
Confidentiality	Encryption	TLS, disk encryption	Brute force, cryptanalysis
Integrity	Hash, MAC	File checks, signatures	Collision attack
Authentication	Certificates	PKI, SSL/TLS	MITM, fake certificates
Nonrepudiation	Digital signature	Signed email, code	Hash collision forgery

Study order that matches your videos: concepts and algorithms (sections 3.1 to 3.12), PKI and protocols (3.13 to 3.25), attacks (3.26 to 3.43), defense (3.44 to 3.46).

3. Topic-by-Topic Guide

3.1 Cryptography and Its Four Objectives

Source: page 1, slide 4.

What it is. Cryptography is the science of turning readable data into scrambled data so only the right people can read or trust it. The readable data is called **plaintext**. The scrambled result is called **ciphertext**. A **key** is the secret value that controls the scrambling. **Encryption** turns plaintext into ciphertext. **Decryption** turns it back. The notes say cryptography protects email messages, chat sessions, web transactions, personal data, corporate data, and e-commerce applications, whether sent over a private or a public network.

Objective	Meaning in one line	What provides it
Confidentiality	Only allowed people can read the data	Encryption
Integrity	The data was not changed	Hash, MAC, signature
Authentication	Proof of who sent or owns it	Certificates, signatures
Nonrepudiation	Sender cannot deny sending it	Digital signature

Analogy. A letter in a locked box. The lock is the algorithm, the key is the key, a wax seal proves nobody opened it (integrity), and the sender's personal stamp proves who sent it (authentication and nonrepudiation).

How it works.

1. The sender has plaintext, for example "Dear John, This is my A/C number 7974392830".
2. An algorithm plus a key turns it into ciphertext (random-looking symbols).
3. The ciphertext crosses the network. An eavesdropper sees only noise.
4. The receiver uses the matching key to turn it back into plaintext.

Real-world use. A shop sends card details to a bank. Encryption hides the numbers (confidentiality), a hash check proves the amount was not altered (integrity), and the bank's certificate proves the server is real (authentication).

Countermeasures and defense. Use standard, well-reviewed algorithms with long keys and protect the keys. Details are in sections 3.44 to 3.46.

EXAM FOCUS. The four objectives are listed exactly as Confidentiality, Integrity, Authentication, Nonrepudiation. A question that says "sender cannot deny" means nonrepudiation. "Data was not altered" means integrity.

> **TRAP:** Availability is part of the CIA triad but is NOT one of the four cryptography objectives on this slide. Hashes give integrity, not secrecy.

Memory trick. CAIN: Confidentiality, Authentication, Integrity, Nonrepudiation.

3.2 Symmetric vs Asymmetric Encryption

Source: page 1, slide 4 (types of cryptography).

What it is. Two families of encryption.

- **Symmetric encryption** (also called secret-key, shared-key, or private-key encryption) uses the same key to encrypt and decrypt.
- **Asymmetric encryption** (public-key encryption) uses two different keys. A **public key** encrypts and a mathematically linked **private key** decrypts.

Analogy. Symmetric is a house key: the same key locks and unlocks, so you must hand copies to trusted people safely. Asymmetric is a mailbox: anyone can drop a letter in (public key), but only the owner with the mailbox key (private key) can take letters out.

How it works.

Symmetric:

1. Sender and receiver agree on one secret key beforehand.
2. Sender encrypts with the key. Receiver decrypts with the same key.
3. Problem: how do you share the key safely? With n users you need n(n-1)/2 keys.

Asymmetric:

1. The receiver creates a key pair and publishes the public key.
2. The sender encrypts with the receiver's public key.
3. Only the receiver's private key can decrypt. No secret is shared in advance.

Diagram: the two flows from the slide

```
SYMMETRIC
Plain text --[Encrypt: secret key]--> Cipher text
Cipher text --[Decrypt: SAME secret key]--> Plain text

ASYMMETRIC
Plain text --[Encrypt: PUBLIC key]--> Cipher text
Cipher text --[Decrypt: PRIVATE key]--> Plain text
```

Point	Symmetric	Asymmetric
Keys	One shared key	Public and private pair
Speed	Fast	Slow (about 1000 times slower, verify)
Main problem	Key distribution	Speed and key size
Best for	Bulk data	Key exchange, signatures
Examples	AES, DES, RC4	RSA, ECC, DH

Real-world use. A browser uses asymmetric cryptography to agree on a session key, then uses fast symmetric encryption (AES) for the page data. This mix is called **hybrid encryption**.

Tools and commands used here. OpenSSL (section 5.1), GnuPG (5.6).

Countermeasures and defense. Protect symmetric keys in an HSM or TPM. Authenticate public keys with certificates so an attacker cannot substitute theirs.

EXAM FOCUS. "Same key both ways" -> symmetric. "Public key encrypts, private key decrypts" -> asymmetric. "Key distribution problem" -> symmetric. "Digital signatures and key exchange" -> asymmetric.

> **TRAP:** The slide calls symmetric keys "private-key" encryption. A "private key" in asymmetric encryption is one half of a pair. Read the context: if only one key exists, it is symmetric.

Memory trick. Symmetric = **S**ame key. Asymmetric = **A**part keys (a pair). Public **locks**, private **unlocks**.

3.3 Ciphers: Classical and Modern

Source: page 2, slide 5.

What it is. A **cipher** is the algorithm used to encrypt or decrypt data. The notes split ciphers into two groups.

Classical ciphers (pen-and-paper era):

- **Substitution cipher:** each unit of plaintext is replaced with another symbol. Example: Caesar cipher shifts each letter by a fixed number (ADDED CONTEXT).
- **Transposition cipher:** the plaintext letters are shifted about (rearranged) to form the **cryptogram** (the scrambled message). The letters stay the same, only their order changes. Example: rail fence (ADDED CONTEXT).

Modern ciphers are grouped two ways:

- By key type: **private key** (same key both ways) and **public key** (two different keys).
- By input data: **block cipher** (encrypts fixed-size blocks) and **stream cipher** (encrypts a continuous stream bit by bit or byte by byte).

Analogy. Substitution is swapping every word in a sentence with a code word. Transposition is cutting the sentence into pieces and shuffling the pieces. A block cipher is a machine that takes a box of 16 bytes at a time. A stream cipher is a tap that scrambles water as it flows.

How it works (block vs stream).

1. Block cipher: split the data into blocks (for example 128 bits), pad the last block, and encrypt each block with the key using a **mode**.
2. Stream cipher: generate a pseudo-random **keystream** from the key and XOR it with the data bit by bit.

> **ADDED CONTEXT:** Block cipher modes (the OpenSSL list in section 3.17 shows them). ECB: each block encrypted alone, identical plaintext blocks give identical ciphertext, so patterns leak (weak). CBC: each block is XORed with the previous ciphertext block, needs an IV (random start value). CFB and OFB: turn a block cipher into a stream-like cipher. CTR: encrypts a counter and XORs the result with the data. GCM: CTR plus built-in integrity check (authenticated encryption).

Real-world use. AES-CBC or AES-GCM protects a file. ChaCha20 protects a phone's HTTPS traffic. A Caesar cipher appears only in puzzles and exam questions.

AI angle. An AI assistant can identify a classical cipher from a sample and run frequency analysis. Example prompt: "Here is a ciphertext from an authorized CTF. List the likely classical ciphers and the checks that separate them." Frequency analysis is covered in 3.28.

Countermeasures and defense. Never use ECB for real data. Never reuse an IV or nonce with the same key. Prefer authenticated modes (GCM, ChaCha20-Poly1305).

EXAM FOCUS. "Letters rearranged" -> transposition. "Letters replaced" -> substitution. "Fixed-size chunks" -> block. "Continuous stream" -> stream. The classical cipher names are rarely asked beyond this.

> **TRAP:** Transposition does not change the letters. Substitution does not change the positions.

Memory trick. **S**ubstitute = **S**wap letters. **T**ranspose = **T**ransfer positions. RC4 is the stream cipher to remember; AES is the block cipher to remember.

3.4 Symmetric Encryption Algorithms

Source: page 2, slide 6.

What it is. A catalog of 17 symmetric algorithms. For each the exam cares about three facts: block or stream, key size, and block size.

Algorithm	Cipher type	Key size (bits)	Block size (bits)	Application areas
DES	Block	56	64	Legacy systems, early standards
3DES	Block	112, 168	64	Financial services, payments
AES	Block	128, 192, 256	128	Secure comms, storage, government
RC4	Stream	40 to 2048 (variable)	-	HTTPS, WEP/WPA, streaming

Algorithm	Cipher type	Key size (bits)	Block size (bits)	Application areas
RC5	Block	0 to 2040 (variable)	32, 64, 128	Crypto libraries, secure comms
RC6	Block	128, 192, 256	128	AES competition finalist
Blowfish	Block	32 to 448 (variable)	64	DES replacement, secure storage
Twofish	Block	128, 192, 256	128	File and disk encryption
IDEA	Block	128	64	Secure email (PGP), data encryption
Threefish	Block	256, 512, 1024	256, 512, 1024	Disk encryption (Skein hash)
Serpent	Block	128, 192, 256	128	High security, AES finalist
Camellia	Block	128, 192, 256	128	Secure comms, Japanese standard
TEA	Block	128	64	Lightweight, embedded systems
CAST-128	Block	40 to 128	64	Software apps, secure comms
CAST-256	Block	128, 160, 192, 224, 256	128	Advanced encryption, libraries
ChaCha20	Stream	256	-	Secure comms, modern protocols
Salsa20	Stream	256	-	Secure comms, crypto protocols

> **ADDED CONTEXT:** Extra facts that help with elimination (verify details before relying on them for numbers).

Algorithm	Origin	Fact worth knowing
DES	IBM, US standard 1977	Feistel, 16 rounds, 56 bits + 8 parity
3DES	DES applied 3 times	168-bit key, only 112-bit strength
AES	Rijndael (Daemen, Rijmen)	10, 12, 14 rounds; FIPS 197

Algorithm	Origin	Fact worth knowing
RC4	Ron Rivest	Broken, banned in TLS, WEP flaw
RC5, RC6	Ron Rivest	RC6 was an AES finalist
Blowfish	Bruce Schneier	Basis of bcrypt; Twofish is successor
IDEA	Lai and Massey	Used by early PGP
Threefish	Schneier et al.	Core of Skein, a SHA-3 finalist
Serpent	Anderson, Biham, Knudsen	AES finalist, very conservative
Camellia	Mitsubishi and NTT	Similar strength to AES
TEA	Wheeler and Needham	Tiny code, weak key schedule
CAST-128	Adams and Tavares	Old PGP/GPG default
ChaCha20	Daniel Bernstein	Used with Poly1305 in TLS 1.3

The five AES competition finalists were MARS, RC6, Rijndael, Serpent, and Twofish. Rijndael won and became AES.

Analogy. Think of a car lot. DES is a rusty old car (works, but unsafe). 3DES is the same car with extra patches. AES is the modern standard car. ChaCha20 is a fast new model that runs well on phones.

How it works (what "key size" and "block size" mean).

1. Key size is the length of the secret. Longer keys mean more possible keys, so brute force takes longer.
2. Block size is the chunk the cipher processes each step. A 64-bit block is small enough that very large data sets can leak patterns (the Sweet32 attack, ADDED CONTEXT). A 128-bit block is safer.
3. Stream ciphers have no block size because they work on a continuous keystream.

Real-world use. A bank still running 3DES on old card terminals upgrades to AES-256 to meet policy. A VPN picks ChaCha20 for mobile clients with no AES hardware.

Tools and commands used here. OpenSSL `enc` (section 5.1), VeraCrypt (3.24), GnuPG (3.19).

AI angle. An AI code assistant can scan source code for weak ciphers. Example prompt: "Review this Java code for deprecated algorithms (DES, 3DES, RC4, ECB mode) and propose AES-GCM replacements." Always review the output, since assistants can also suggest insecure defaults.

Countermeasures and defense. Retire DES, RC4, and 3DES. Use AES-128 or AES-256, or ChaCha20. For symmetric algorithms prefer 256-bit keys (section 3.44).

EXAM FOCUS. Questions give a number and ask for the algorithm: "56-bit key" -> DES. "168-bit" or "three passes" -> 3DES. "128/192/256 and 128-bit blocks" -> AES. "Variable key up to 2048 bits, stream" -> RC4. "Key 32 to 448 bits" -> Blowfish. "256, 512, 1024" -> Threefish. "Stream, 256-bit key" -> ChaCha20 or Salsa20.

> **TRAP:** DES uses a 64-bit key field but only 56 bits are real key bits. The exam answer is 56. 3DES lists both 112 and 168 because of two-key and three-key options.

Memory trick. 64-bit blocks are the old guard: **D**ES, **T**riple DES, **B**lowfish, **I**DEA, **T**EA, **C**AST-128 ("Don't Try Boiling Instant Tea, Cats"). 128-bit blocks are the AES family: AES, RC6, Twofish, Serpent, Camellia, CAST-256. Streams: RC4, ChaCha20, Salsa20.

3.5 Asymmetric Encryption Algorithms

Source: page 3, slide 7.

What it is. Five public-key algorithms, each with a role.

Algorithm	Key size (bits)	Application area	Hard problem it relies on
RSA	Variable	Encryption, signatures, key exchange	Factoring large numbers
DSA	Variable	Digital signatures only	Discrete logarithm
Diffie-Hellman	Variable	Key exchange	Discrete logarithm
ECC	160-521	Encryption, signatures, key exchange	Elliptic-curve discrete log
ElGamal	Variable	Encryption, key exchange	Discrete logarithm

The last column is ADDED CONTEXT. RSA stands for **Rivest-Shamir-Adleman** (1977). Diffie-Hellman (1976) only agrees on a shared secret and does not encrypt messages itself. **ECC** (Elliptic Curve Cryptography) gives similar strength to RSA with much smaller keys, which suits phones and IoT. DSA signs but cannot encrypt.

> **ADDED CONTEXT:** NIST SP 800-57 strength equivalents (verify).

Strength (bits)	Symmetric	RSA or DH	ECC
112	3DES	2048	224
128	AES-128	3072	256
192	AES-192	7680	384
256	AES-256	15360	512 or more

Analogy. RSA is a padlock factory: anyone can snap a padlock shut (public key), but only the factory owner holds the key (private key). It is easy to multiply two big primes but extremely hard to split the product back, like mixing paint is easy but unmixing is not.

How it works (RSA in plain steps).

1. Pick two large secret primes p and q. Multiply them to get n.
2. Build a public key (n, e) and a private key d derived from p and q.
3. Encrypt with the public key. Only d can decrypt.
4. To sign, reverse the roles: transform the hash with the private key. Anyone verifies with the public key.

Real-world use. A website's certificate holds an RSA or ECC public key. The browser uses it to protect the TLS key exchange (TLS 1.2) or to check the server's signature (TLS 1.3).

Tools and commands used here. OpenSSL, GnuPG, RsaCtfTool (attacks weak RSA keys, section 3.43).

AI angle. Attackers do not "AI-crack" RSA. AI can help find weak implementations, for example reused primes or short keys, by scripting checks. Example prompt: "Write a Python script that checks whether two RSA public keys share a prime factor (GCD test)."

Countermeasures and defense. Use RSA of at least 2048 bits (slide 52 item 8). Prefer ECC P-256 or higher. Authenticate DH exchanges, since unauthenticated DH falls to a man-in-the-middle attack (3.27). Plan for post-quantum algorithms (3.45).

EXAM FOCUS. "Signatures only" -> DSA. "Key exchange only" -> Diffie-Hellman. "Smaller keys, same strength, mobile" -> ECC. "Factoring" -> RSA. All five break under a large quantum computer (Shor's algorithm, ADDED CONTEXT).

> **TRAP:** Diffie-Hellman does not encrypt data and does not authenticate. DSA does not encrypt.

Memory trick. **R**SA = **R**oles for everything (encrypt, sign, exchange). **D**SA = **D**igital signature only. **D**H = **D**eal on a key. **E**CC = **E**conomy-size keys. ElGamal = enc plus key exchange.

3.6 Message Digest (One-Way Hash) Functions

Source: page 3, slide 8.

What it is. A **hash function** takes any amount of data and produces a fixed-size value called a **message digest** (hash value). The notes give three properties:

- The output has a fixed size for any input.

- If any input bit changes, every output bit has a 50 percent chance of changing. This is the **avalanche effect**.
- It is computationally infeasible to find two files with the same digest.

Digests are called **one-way hash functions** because they cannot be reversed. The slide example is `a14092af948b938569584e5b8d8d307a` (32 hex characters = 128 bits, the size of an MD5 digest).

Analogy. A fingerprint. A fingerprint identifies a person, but you cannot rebuild the person from it. Change one hair of the person and a totally different fingerprint appears.

How it works.

1. Input (a document) goes into the hash function.
2. The function processes it in blocks through rounds of mixing.
3. A fixed-length digest comes out.
4. The sender sends data plus digest. The receiver recomputes the digest and compares. A match means the data was not changed.

Real demo (real output from a Linux shell):

```
printf 'hello' | md5sum
printf 'Hello' | md5sum
```

```
5d41402abc4b2a76b9719d911017c592  -
8b1a9953c4611296a827abf8c47804d7  -
```

Changing one letter (h to H) gives a completely different digest. That is the avalanche effect.

Algorithm	Digest length (bits)	Length in hex characters
MD5	128	32
SHA-1	160	40
SHA-256	256	64
SHA-512	512	128

Real-world use. A download page lists a SHA-256 value. After download, the user hashes the file. If the values match, the file is intact. Systems also store password hashes instead of passwords.

Tools and commands used here. md5sum, sha256sum (section 5.5), the calculators in 3.8, CyberChef (3.9), hashcat (3.30).

AI angle. Given a hash string, an AI assistant can narrow the likely algorithm by length and format. Example prompt: "I have this 40-character hex value from an

authorized assessment. Which hash types could it be, and which hashcat modes should I test?" Tools such as hashid also do this.

Countermeasures and defense. For integrity use SHA-256 or higher (slide 52 item 9). For passwords use salted, slow functions (3.46), never plain MD5 or SHA-1.

EXAM FOCUS. "Cannot be reversed" -> one-way hash. "Fixed length output whatever the input" -> hash. "Detect tampering" -> hash. "Every output bit 50 percent" -> avalanche effect.

> **TRAP:** A hash is not encryption. There is no key and no decryption. "Cracking" a hash means guessing inputs until one matches.
>
> **ADDED CONTEXT:** Three attack-resistance properties: preimage resistance (given a hash, cannot find an input), second-preimage resistance (given an input, cannot find another with the same hash), and collision resistance (cannot find any two inputs with the same hash). Collisions must exist (more inputs than outputs). The goal is that finding one is infeasible.

Memory trick. Hash = **H**ash browns: once chopped, you cannot un-chop. Length by letter: MD**5** = 128 bits = 32 hex, SHA-**1** = 160 bits = 40 hex.

3.7 Message Digest Function Comparison (MD, SHA, and Others)

Source: page 4, slide 9.

What it is. A comparison of 13 hash algorithms. The original is a wide table; here it is split into two tables with the same data.

Algorithm	Output (bits)	Internal state (bits)	Block (bits)	Max message (bits)
MD2	128	128	128	2^64
MD4	128	128	512	2^64
MD5	128	128	512	2^64
MD6	224, 256, 384, 512	1024	512	Unlimited
SHA-0	160	160	512	2^64
SHA-1	160	160	512	2^64
SHA-2	224, 256, 384, 512	256, 512	512, 1024	2^128
SHA-3	224, 256, 384, 512	1600	1088, 576	Unlimited
RIPEMD-160	160	160	512	2^64
WHIRLPOOL	512	512	512	2^256

Algorithm	Output (bits)	Internal state (bits)	Block (bits)	Max message (bits)
Tiger	192	192	512	Unlimited
BLAKE2	256, 512	256, 512	512, 1024	Unlimited
BLAKE3	256	256	512	Unlimited

Algorithm	Rounds	Operations	Security (bits)	Application areas
MD2	18	Permutation, substitution	128	Legacy, checksums
MD4	48	AND, OR, XOR	64	Obsolete early hash
MD5	64	AND, OR, XOR	64	File checks, checksums, signatures
MD6	Variable	AND, OR, XOR	128-256	Data integrity
SHA-0	80	Bitwise logic	0	Obsolete, replaced by SHA-1
SHA-1	80	Bitwise logic	80	Legacy, software updates, TLS
SHA-2	64, 80	AND, OR, XOR	112-256	Secure apps, signatures, SSL
SHA-3	Variable	Sponge construction	112-256	Next-generation hashing
RIPEMD-160	160	AND, OR, XOR	80	Data integrity
WHIRLPOOL	10	Matrix ops, substitution	256	Secure hashing
Tiger	24	AND, OR, XOR	192	High-speed, checksums
BLAKE2	10-14	AND, OR, XOR	128-256	High-speed hashing
BLAKE3	Variable	AND, OR, XOR	128	High-performance hashing

How to read "Security (bits)". It is the work factor of the best known attack, in powers of 2. MD5 lists 64 because a collision attack needs about 2^(128/2) = 2^64 steps by the birthday method (ADDED CONTEXT). SHA-0 lists 0 because practical collisions exist. A higher number means a stronger hash.

Analogy. A ladder of safes. MD2 and MD4 are cardboard boxes, MD5 and SHA-1 are cracked steel safes, SHA-2 and SHA-3 are current vaults.

ADDED CONTEXT: Status summary (verify dates). MD5: collisions since 2004, forged CA certificate 2008, Flame malware abused an MD5 collision in 2012. SHA-1: first public collision (SHAttered) in 2017. SHA-2: no practical break. SHA-3: Keccak sponge, standardized 2015 (FIPS 202), a different design so one break does not affect both. RIPEMD-160 is used in Bitcoin address creation. BLAKE2 and BLAKE3 are fast modern hashes (BLAKE3 uses a tree design). MD6 was a SHA-3 candidate that was not chosen.

Real-world use. Software vendors publish SHA-256 sums. Git used SHA-1. Bitcoin uses SHA-256 and RIPEMD-160. A forensics tool uses MD5 plus SHA-1 or SHA-256 to prove evidence was not altered.

Countermeasures and defense. Replace MD5 and SHA-1 with SHA-256 or SHA-3 for signatures and integrity. Use salted slow hashing for passwords.

EXAM FOCUS. Digest size: MD2, MD4, MD5 = 128; SHA-0, SHA-1, RIPEMD-160 = 160; Tiger = 192; WHIRLPOOL = 512; SHA-2 = 224/256/384/512; SHA-3 uses the **sponge construction** with a 1600-bit state. Rounds: MD5 = 64, SHA-1 = 80, MD4 = 48, MD2 = 18, WHIRLPOOL = 10. "Replaced SHA-0" -> SHA-1.

TRAP: SHA-2 and SHA-3 are not "SHA-2 version 2". SHA-3 has a new internal design (sponge), not the SHA-2 round structure.

Memory trick. "MD two-four-five: 128 all, MD4 48 rounds, MD5 64." And "SHA-**1** has **1**60 bits, **80** rounds". Order of safety: MD < SHA-1 < SHA-2 < SHA-3.

3.8 Message Digest Function Calculators

Source: page 4, slide 10.

What it is. Free tools (desktop and web) that compute hash values so you can check file integrity.

Tool	What it does	Type
HashMyFiles (nirsoft.net)	Shows MD5, SHA-1, and more for files	Windows desktop
MD5 Calculator (bullzip.com)	Computes MD5 for files and text	Windows desktop
MD6 Hash Generator (browserling.com)	Generates MD6 hashes online	Web
All Hash Generator (browserling.com)	Many hash types for one input	Web
md5 hash calculator (onlinehashtools.com)	Online MD5 calculator	Web
Message Digester (freeformatter.com)	Digests text with many algorithms	Web

Tool	What it does	Type
MD6 Hash Generator (atatus.com)	Generates MD6 hashes online	Web

Analogy. A kitchen scale for files: it does not change the file, it only gives a number you can compare.

How it works.

1. Select a file or paste text.
2. Pick the algorithm (MD5, SHA-1, and so on).
3. Read the digest and compare it with the published value.

Real-world use. A tester downloads a tool, hashes it with HashMyFiles, and compares the result with the vendor's SHA-256 to rule out tampering.

> **TRAP:** Never paste confidential data into a web hash site. The site receives your input.

Countermeasures and defense. Download over HTTPS and compare against a hash obtained through a separate trusted channel. Prefer SHA-256 over MD5 for checks.

EXAM FOCUS. The exam may list these names and ask "which tool computes message digests?" HashMyFiles is the Windows file tool. Browser tools handle MD6 because common OS tools do not.

Memory trick. Hash**MyFiles** = hash **my files** (NirSoft, local files).

3.9 Multi-Layer Hashing Calculators (CyberChef)

Source: page 5, slide 11.

What it is. Multi-layer hashing (also called **nested hashing** or **recursive hashing**) applies a hash function more than once: to the input, then to the previous output. **CyberChef** (from GCHQ, a free browser tool) can chain these steps in a "recipe".

The slide shows a recipe titled "2 MD5, 3 SHA1, HMAC - CyberChef" with operations HMAC, MD5, SHA1 (Rounds 80), and HMAC (Key 12 HEX). Input is `My Account number is 0214569150` (31 bytes, file name Secret.txt). Output is `bcabf7b4f05f95e7862933e911904b01`.

[UNCLEAR on page 5] The exact order of operations and the HMAC key text were small in the screenshot. The output is 32 hex characters (128 bits), consistent with MD5 or HMAC-MD5 as the final step.

Analogy. Photocopying a photocopy. Each layer adds a transformation, so anyone who wants to reproduce the final value must know every step.

How it works.

1. Choose input text or a file in CyberChef.
2. Drag operations into the recipe: hash, then hash again, or add HMAC.
3. CyberChef runs each step on the previous output.
4. The final value appears in the Output pane.

> **ADDED CONTEXT: HMAC** (Hash-based Message Authentication Code) mixes a secret key into a hash, so it proves both integrity and that the sender knows the key. A plain hash proves only integrity. Nested hashing adds work for attackers but is NOT a substitute for a proper slow password hash such as bcrypt or PBKDF2.

Real-world use. An analyst reproduces a vendor's custom checksum for a configuration file by chaining the same operations in CyberChef.

AI angle. An AI assistant can describe what a CyberChef recipe does or write the equivalent Python. Example prompt: "Explain what HMAC-MD5 then SHA1 does to this input, and write Python hashlib code that reproduces it."

Countermeasures and defense. Do not invent chained hashes for passwords. Use a vetted KDF (3.46).

EXAM FOCUS. "Hash applied multiple times" -> multi-layer, nested, or recursive hashing. "Browser tool for chaining operations" -> CyberChef.

Memory trick. Cyber Chef = a chef who cooks data in layers, one operation per step.

3.10 Hardware-Based Encryption

Source: page 5, slide 12.

What it is. Hardware-based encryption uses dedicated hardware to help or replace software during encryption. These devices can also store keys and other sensitive data in protected areas of RAM or in nonvolatile storage, away from the normal operating system.

Device	What it is (from the notes)	Plain meaning
TPM	Trusted Platform Module, crypto-processor chip on the motherboard	Built-in vault chip for keys and boot checks
HSM	Hardware Security Module, external device for crypto-processing	Dedicated key-safe box or card
USB encryption	Onboard encryption for USB storage devices	Flash drive that encrypts itself

Device	What it is (from the notes)	Plain meaning
Hard drive encryption	Data on the drive encrypted with a range of options	Drive that scrambles what it stores

Analogy. Software encryption is hiding your valuables under a rug in your house. A TPM is a safe bolted to the floor. An HSM is a bank vault you rent. A self-encrypting USB is a lockbox that travels with you.

How it works.

1. The device generates or receives a key and stores it inside secured memory. The key never leaves in plain form.
2. The OS or application sends data to the device.
3. The device performs the encryption, decryption, or signing inside itself and returns only the result.
4. If someone steals the disk or the device, the keys stay locked inside.

Real-world use. A laptop with BitLocker keeps the disk key in the TPM, so a stolen drive moved to another PC stays unreadable. A bank keeps the private signing key of its CA inside an HSM.

> **ADDED CONTEXT:** TPM is on the motherboard, one per machine, and also records boot measurements (used by secure boot and BitLocker). An HSM is shared by many servers, often validated to FIPS 140-2 or 140-3 and used for CA keys, payment PINs, and code signing. Self-encrypting drives (SEDs) follow the TCG Opal standard.

Tools and commands used here. BitLocker (3.24), HSM appliances, TPM.

Countermeasures and defense. Use TPM plus a PIN for full-disk encryption. Keep HSM firmware updated and back up keys under dual control. Slide 54 item 8 and 10 recommend quantum-ready TPMs and HSMs.

EXAM FOCUS. "Chip on the motherboard" -> TPM. "External device that manages and stores keys" -> HSM. "Generates, manages, securely stores cryptographic keys" -> HSM.

> **TRAP:** TPM is internal and fixed to the board. HSM is an additional external (or add-in) device. Do not swap them.

Memory trick. **T**PM = **T**ied to the board. **H**SM = **H**ired box that serves many.

3.11 Quantum Cryptography

Source: page 6, slide 13.

What it is. Quantum cryptography uses quantum mechanics to protect keys. The best known method is **quantum key distribution (QKD)**. The notes say data is carried by

a sequence of **photons** (light particles) with a spinning (polarization) trait. As the photons travel, they pass through filters: vertical, horizontal, forward slash, and backslash. The notes state that attackers can eavesdrop but cannot manipulate the data because the photons pass through arbitrary filters.

Analogy. Imagine sending coins in sealed tubes that change if anyone peeks. If the receiver finds the coins changed, they know someone looked.

How it works.

1. The sender sends photons, each polarized by a randomly chosen filter.
2. The receiver measures each photon with randomly chosen filters.
3. They compare (over a normal channel) which filter types they used, not the results. They keep results where the filters matched. These bits become the shared key.
4. If an eavesdropper measured the photons, the measurement disturbs them, which raises the error rate. The pair sees the errors and discards the key.

> **ADDED CONTEXT:** The exact method above is the BB84 protocol (Bennett and Brassard, 1984). Standard QKD theory says eavesdropping is detected (it disturbs the photons), while the slide's phrase "cannot manipulate" is a simplified summary. If a question quotes the slide, answer with the slide's wording. QKD only distributes keys. It does not encrypt bulk data; a symmetric cipher does that afterwards.

Real-world use. Banks and government networks in some countries run QKD links between data centers over dedicated fiber.

Countermeasures and defense. QKD needs authenticated classical channels, tamper-proof devices, and protection from side-channel attacks on detectors. Section 3.42 lists quantum-era attacks.

EXAM FOCUS. "Photons, polarization, filters (vertical, horizontal, forward slash, backslash)" -> quantum cryptography. "Quantum key distribution" -> QKD. It is based on quantum mechanics, not on math problems like factoring.

> **TRAP:** Quantum cryptography (QKD) is a defense. Quantum cryptanalysis (3.26) is an attack by a quantum computer. They are different things.

Memory trick. **Q**KD = **Q**uiet **K**ey **D**elivery by light: peeking leaves a mark.

3.12 Cryptography Tools

Source: page 6, slide 14.

What it is. Software for encrypting text and files.

Tool	What it does	Platform	Exam keyword
BCTextEncoder (jetico.com)	Encrypts text with passwords or public keys	Windows	Encoded message block
CryptoForge	Encrypts files, folders, and email	Windows	File encryption suite
AxCrypt	Encrypts files with right-click, AES	Windows, macOS	File encryption, simple
Microsoft Cryptography Tools	Microsoft utilities for certificates and keys	Windows	Microsoft crypto utilities
Concealer	Encrypts files into protected cards	macOS	Mac file vault
SensiGuard	File encryption and secure deletion	Windows	File security tool

BCTextEncoder in detail (slide 14 screenshot). The utility is version 1.03.21 in the screenshot. It compresses plain text, encrypts it, and converts it into a text block between `-----BEGIN ENCODED MESSAGE-----` and `-----END ENCODED MESSAGE-----` that you can paste into any email or chat. The screenshot shows "Decoded plain text: 282 B", "Encoded text: 622 B", and "Encoded by password" (the sender chose password mode instead of a public key). The notes say it uses strong symmetric and public-key algorithms. [UNCLEAR on page 6] The small text in the screenshot was hard to read, so treat the byte counts as best reading.

[UNCLEAR on page 6] The slide shows a jetico.com link without a label. BCTextEncoder is made by Jetico, so the link belongs to BCTextEncoder.

Analogy. A text scrambler: type a message, press a button, and get a safe block of gibberish that only someone with the password can unscramble.

How it works.

1. Type or paste text in the tool.
2. Choose password mode or public-key mode.
3. Encode. Copy the encoded block into your email.
4. The receiver pastes the block into the tool and decodes it.

Real-world use. Two colleagues exchange a client's account details in a chat. They encode the text with a password sent by phone.

Countermeasures and defense. Send passwords through a separate channel. Use long passphrases, since password mode relies on the passphrase.

EXAM FOCUS. "Encrypts confidential text in a message with symmetric and public-key algorithms" -> BCTextEncoder.

Memory trick. BC + **Text** + **Encoder** = encodes your text for safe pasting.

3.13 Public Key Infrastructure (PKI)

Source: page 7, slide 16.

What it is. PKI is the set of hardware, software, people, policies, and procedures needed to create, manage, distribute, use, store, and revoke digital certificates. A **digital certificate** is an electronic ID card that binds a name to a public key. Example CAs shown: DigiCert, Comodo, GoDaddy.

Component	Role in the notes	Plain meaning
Certificate Management System	Generates, distributes, stores, verifies certificates	The back-office software
Digital certificates	Establish credentials in online transactions	The ID cards
Validation Authority (VA)	Stores certificates with their public keys	The status desk
Certificate Authority (CA)	Issues and verifies digital certificates	The passport office
End user	Requests, manages, and uses certificates	The person or server
Registration Authority (RA)	Acts as the verifier for the CA	The identity checker

Analogy. Passports. The RA checks your documents. The CA prints and signs the passport. The VA is the border database that confirms the passport is still valid. Border guards (browsers) trust the passport because they trust the issuing country (the CA).

How it works (the slide's flow).

Diagram: PKI issuing and checking

```
(a) User  ---------> RA     user applies for a certificate
(b) RA    ---------> CA     RA verifies the applicant and forwards
(c) CA    ---------> User   CA issues the public key certificate
(d) CA    ---------> VA     CA updates the validation authority
(e) User  ---------> Shop   signed message plus certificate copy
(f) Shop  ---------> VA     asks if the certificate is valid
(g) VA    ---------> Shop   returns the determined result
```

1. The user applies to the RA.
2. The RA verifies the applicant and requests issuance from the CA.
3. The CA issues a certificate that carries the user's public key and the CA's signature, and updates the VA.
4. The user signs a message with the private key and sends it with the certificate. The shop validates the signature with the public key.
5. The shop queries the VA about certificate validity and gets a result.

Real-world use. A company buys a certificate for its website from a CA. The CA proves the company controls the domain, issues the certificate, and browsers that trust the CA show a padlock.

> **ADDED CONTEXT:** Certificates follow the **X.509** standard and contain: version, serial number, signature algorithm, issuer, validity dates, subject, subject public key, extensions, and the CA's signature. Trust forms a **chain**: root CA -> intermediate CA -> your certificate. Root CAs are kept offline. Browsers ship with a trust store of root CAs.

Tools and commands used here. Java keytool, OpenSSL, Windows certificate manager.

AI angle. Deepfake audio and AI-written documents make identity proofing harder, so RAs rely more on multi-step checks. Defenders use AI to monitor Certificate Transparency logs for look-alike domains (ADDED CONTEXT). Example prompt: "List all certificates issued in the last 7 days for domains that resemble example.com."

Countermeasures and defense. Protect CA private keys in an HSM. Revoke compromised certificates quickly. Check revocation (CRL or OCSP). Pin or monitor certificates for critical sites.

EXAM FOCUS. "Issues certificates" -> CA. "Verifies the applicant for the CA" -> RA. "Stores certificates / answers validity" -> VA. "Set of hardware, software, people, policies, procedures" -> PKI.

> **TRAP:** The RA does not sign or issue certificates. If a question asks who issues, answer CA even if RA is an option that sounds official.

Memory trick. RA **R**eads your documents, **C**A **C**reates the certificate, **V**A **V**erifies it later (R, C, V).

3.14 Signed Certificate (CA) vs Self-Signed Certificate

Source: page 7, slide 17.

What it is. Two ways to get a certificate.

Point	Signed by a CA	Self-signed
Who creates it	A trusted CA after checking you	You, with a tool
Contents	Holder name, serial, expiry, public key, CA signature	User name, public key, own signature
Tools	Bought or issued by a CA	Adobe Acrobat Reader, Java keytool, Apple Keychain
Who verifies it	Receiver asks the VA	Receiver asks the user
Trust	High (third party vouches)	Low (you vouch for yourself)

Analogy. A CA-signed certificate is a government passport. A self-signed certificate is a handwritten ID card you made at home: it may be accurate, but nobody else vouches for it.

How it works.

Signed certificate:

1. The user buys a certificate from a CA.
2. The user signs the document with the private key and sends it.
3. The receiver verifies the certificate by asking the VA.
4. The CA publishes CRLs, OCSP responses, and the CA chain certificate. **CRL** = Certificate Revocation List (list of revoked certificates). **OCSP** = Online Certificate Status Protocol (live status query).

Self-signed certificate:

1. The user creates the certificate with a tool.
2. The user signs the document with it and sends it.
3. The receiver must ask the user to confirm the certificate. There is no third-party check.

Real-world use. A developer uses a self-signed certificate on a test server. A public online shop must use a CA-signed one, or browsers show a warning.

Countermeasures and defense. Do not trust self-signed certificates for public services. Distribute internal self-signed or private-CA roots only through managed channels. An attacker can easily create a self-signed certificate for a MITM attack, so never click through browser warnings.

EXAM FOCUS. "Verified by asking the user" -> self-signed. "VA, CRL, OCSP" -> CA-signed. "Created with Java keytool or Keychain" -> self-signed.

> **TRAP:** Self-signed does not mean encrypted differently. The encryption can be just as strong. What is missing is third-party trust.

Memory trick. Self-signed = **self**ish: it vouches for itself.

3.15 Digital Signature

Source: page 8, slide 18.

What it is. A digital signature uses asymmetric cryptography to simulate a handwritten signature in digital form. It proves who signed (authentication), that the data did not change (integrity), and that the signer cannot deny it (nonrepudiation). The signed message can also be encrypted for confidentiality.

Analogy. A notarized letter in a locked pouch. The notary stamp is the signature. The pouch lock is the encryption.

How it works (the slide's six steps).

Diagram: sign and seal, then open and verify

```
SENDER
 SIGN     hash the message and sign the hash with SENDER PRIVATE key
          append the signed hash to the message
 SEAL     encrypt the message with a one-time symmetric key
          encrypt that symmetric key with RECIPIENT PUBLIC key
 DELIVER  mail the electronic envelope

RECIPIENT
 ACCEPT   receive the envelope
 OPEN     decrypt the symmetric key with RECIPIENT PRIVATE key
          decrypt the message with the symmetric key
 VERIFY   rehash the message and compare with the attached hash
          unlock the attached hash with SENDER PUBLIC key
```

1. **Sign:** hash the message, then sign the hash with the sender's private key and attach it.
2. **Seal:** encrypt the message with a one-time symmetric key, then encrypt that key with the recipient's public key.
3. **Deliver:** send the envelope.
4. **Accept and open:** the recipient decrypts the symmetric key with their private key, then decrypts the message.
5. **Verify:** rehash the message. Decrypt the attached signed hash with the sender's public key. If both hashes match, the message is authentic and unchanged.

Real-world use. A vendor signs a software installer. Windows checks the signature and warns if the file changed or if the signer is unknown.

> **ADDED CONTEXT:** Standard signature algorithms: RSA, DSA, ECDSA, EdDSA. The standard is the Digital Signature Standard (FIPS 186). Only the hash is signed because asymmetric encryption is slow. A digital signature is different from a MAC (HMAC): a MAC uses a shared secret, so it cannot give nonrepudiation.

AI angle. Deepfakes and fake documents create demand for signed provenance. Standards such as C2PA attach signed "content credentials" to media (ADDED CONTEXT). Verifying a signature does not prove the content is true, only who signed it and that it is unchanged.

Countermeasures and defense. Protect private keys (HSM or smart card). Use SHA-256 or better, since MD5 and SHA-1 allow collision forgery (3.33). Check certificate validity before trusting a signature.

EXAM FOCUS. The classic key-use question: **sign with the sender's private key, verify with the sender's public key. Encrypt with the recipient's public key, decrypt with the recipient's private key.**

> **TRAP:** A signature does not hide the message. Without SEAL, the message is readable. And the sender never uses the recipient's private key.

Memory trick. "**P**rivate **P**roves (signs), **P**ublic **P**rotects (encrypts)". Process words: **S**ign, **S**eal, **D**eliver, **A**ccept, **O**pen, **V**erify = "Sam Sends Documents And Opens Vaults".

3.16 Secure Sockets Layer (SSL)

Source: page 8, slide 19.

What it is. SSL is a protocol developed by Netscape to manage the security of message transmission on the Internet. The notes call it an application layer protocol and say it uses RSA asymmetric (public-key) encryption to protect data sent over SSL connections.

> **ADDED CONTEXT:** SSL is old and insecure. SSL 2.0 (1995) and SSL 3.0 (1996) are deprecated. TLS is its successor (3.17). Many references place SSL/TLS between the application and transport layers. If an exam question offers "application layer" for SSL, follow your notes.

Analogy. Two strangers meeting in a lobby. They introduce themselves (Hello), one shows an ID card (certificate), they whisper a secret code to each other (premaster secret), then they walk into a private room (encrypted session).

How it works (the SSL handshake on the slide).

Diagram: SSL handshake

```
CLIENT                                                SERVER
  |----- Client Hello ------------------------------------->|
  |<---- Server Hello + Certificate ------------------------|
  |<---- Server Hello Done ---------------------------------|
  |   (client verifies the certificate and makes premaster)  |
  |----- Client Key Exchange (premaster secret) ------------>|
  |----- Change Cipher Spec + Finished --------------------->|
  |                (server checks the handshake hash)        |
  |<---- Change Cipher Spec + Finished ---------------------|
  |<=========== encrypted application data ================>|
```

1. **Client Hello:** the client sends the SSL version, random data, supported encryption algorithms, session ID, key exchange algorithms, compression algorithms, and MAC algorithms.
2. **Server Hello + Certificate:** the server picks the SSL version and algorithms, sends a session ID and its certificate.

3. **Server Hello Done:** the server says it is finished.
4. **Client Key Exchange:** the client verifies the certificate, generates a random **premaster secret**, encrypts it with the server's public key, and sends it.
5. **Change Cipher Spec + Finished (client):** the client says "from now on I encrypt", and sends a hash of the handshake messages.
6. **Server check:** the server calculates its own hash of the handshake and compares. If equal, the key and cipher suite negotiation succeeded. It sends its own Change Cipher Spec and Finished message.

Real-world use. An older bank website supports SSL 3.0 for legacy customers. A scanner flags it, because attackers can force a downgrade.

Tools and commands used here. OpenSSL (3.17), Nmap `ssl-enum-ciphers` script (ADDED CONTEXT).

Countermeasures and defense. Disable SSL 2.0 and 3.0 on all servers. Use TLS 1.2 or 1.3. Remove weak cipher suites. Keep private keys off servers that still allow old protocols (see DROWN, 3.35).

EXAM FOCUS. "Developed by Netscape" -> SSL. "Premaster secret encrypted with the server's public key" -> client key exchange step. "Finished message = hash of handshake messages."

> **TRAP:** The premaster secret is encrypted with the SERVER's public key. The client does not send its own private key.

Memory trick. "**H**ello, **H**ello, **C**ert, **D**one, **K**ey, **C**hange, **F**inished" = "Hot Hens Cook Dinner, Kids Chew Food."

3.17 Transport Layer Security (TLS)

Source: page 9, slide 20.

What it is. TLS is a protocol that sets up a secure connection between a client and a server and protects the privacy and integrity of data during transmission. It has two parts:

Sub-protocol	Job (from the notes)	Plain meaning
TLS Handshake Protocol	Client and server authenticate each other, select an encryption algorithm, exchange a symmetric key before data	The negotiation
TLS Record Protocol	Provides secured connections with an encryption method	The protected data pipe

[UNCLEAR on page 9] The slide gives "such as DES" as the Record Protocol encryption example. DES is obsolete, so on a real server the method is AES or ChaCha20. In an exam, remember the slide's wording if DES is the choice.

The handshake messages on the slide (in order).

Diagram: TLS handshake message sequence

```
CLIENT                                                SERVER
  |----- Client Hello ---------------------------------->|
  |<---- Server Hello ----------------------------------|
  |<---- Server Certificate ---------------------------|
  |<---- Server Key Exchange ---------------------------|
  |<---- Certificate Request (optional) ----------------|
  |<---- Server Hello Done -----------------------------|
  |----- Client Certificate (optional) ----------------->|
  |----- Client Key Exchange ---------------------------->|
  |----- Certificate Verify (optional) ----------------->|
  |----- [Change Cipher Spec] --------------------------->|
  |----- Client Finished Message ------------------------>|
  |<---- [Change Cipher Spec] -------------------------------|
  |<---- Server Finished Message ----------------------------|
  |<============ Application Data (both ways) ===========>|
```

The layer view on the slide: Handshake Protocol sits on top of Record Protocol. Application Data is carried inside the Record Protocol.

Analogy. The handshake is two people agreeing which language and secret code to use. The record protocol is the actual encrypted conversation using that code.

How it works.

1. Client and server run the handshake. The server proves identity with a certificate. The client may also present a certificate (optional, mutual authentication).
2. They agree on a cipher suite and exchange or derive a symmetric key.
3. Both send Finished messages to prove the handshake was not tampered with.
4. The Record Protocol encrypts and protects each chunk of application data using that key.

> **ADDED CONTEXT:** TLS 1.0 (1999), 1.1 (2006), 1.2 (2008), 1.3 (2018, RFC 8446). TLS 1.3 removes weak options, makes forward secrecy mandatory, and shortens the handshake. Famous protocol attacks: POODLE (SSLv3), BEAST, CRIME, FREAK, Logjam, Sweet32 (64-bit blocks), DROWN (3.35). **Heartbleed** (2014) was a bug in the OpenSSL library, not in the TLS protocol itself.

OpenSSL on the same slide. The side box reads: OpenSSL is an open-source cryptography toolkit implementing SSL v2/v3 and TLS v1 network protocols and the related cryptography standards required by them (https://www.openssl.org). The terminal on the slide (Parrot Linux, user `attacker`) runs `openssl enc -ciphers` and prints supported ciphers such as aes-128-cbc, aes-128-cfb, aes-128-ctr, aes-128-ecb,

aes-128-ofb, aes-192-..., aes-256-..., aria-128-..., bf, bf-cbc, bf-ecb, bf-ofb, and blowfish, plus aes128-wrap, aes192-wrap, aes256-wrap. Section 5.1 shows how to read this output.

Real-world use. Every HTTPS page you open uses TLS. Port 443 carries HTTP over TLS.

Tools and commands used here. `openssl enc -ciphers`, `openssl s_client` (ADDED CONTEXT), Nmap, SSL Labs test (ADDED CONTEXT).

Countermeasures and defense. Enforce TLS 1.2 or higher, disable old versions and weak ciphers, use certificates from a trusted CA, enable HSTS (ADDED CONTEXT), and patch OpenSSL.

EXAM FOCUS. "Two protocols: Handshake and Record" -> TLS. "Client and server authenticate, select algorithm, exchange symmetric key" -> Handshake. "Provides secured connection with an encryption method" -> Record.

> **TRAP:** The handshake does not carry the web page. The Record Protocol does.

Memory trick. **H**andshake = **H**ello and keys. **R**ecord = **R**eally protects the data.

3.18 Pretty Good Privacy (PGP)

Source: page 9, slide 21.

What it is. PGP is a protocol used to encrypt and decrypt data and to provide authentication and cryptographic privacy. It is used for data compression, digital signing, encrypting and decrypting messages, emails, files, and directories, and to improve email privacy. It combines the best features of conventional (symmetric) and public-key cryptography, so it is a **hybrid cryptosystem**.

Analogy. A courier delivers a big heavy safe (the message locked with a quick, simple lock) and posts the small key to that lock inside a tiny box that only the receiver can open.

How it works.

Diagram: PGP encryption and decryption

```
ENCRYPT (sender)
 Data --[encrypt with random key]--> Encrypted message
 Random key --[encrypt with receiver PUBLIC key RSA]--> Encrypted key
 Send both

DECRYPT (receiver)
 Encrypted key --[decrypt with receiver PRIVATE key RSA]--> Random key
 Encrypted message --[decrypt with random key]--> Data
```

1. PGP generates a one-time random key (the session key). The slide example shows `GSkAQk49fPD2h`.
2. It encrypts the data with that key (fast, symmetric).
3. It encrypts the random key with the receiver's public key (RSA). The slide shows the result `Td7YuEkLg99Qd0`.
4. The sender sends the encrypted message and the encrypted key.
5. The receiver decrypts the random key with their private key, then decrypts the data with it.

Real-world use. A journalist encrypts a source's document with the source's public key before emailing it.

> **ADDED CONTEXT:** PGP was written by Phil Zimmermann in 1991. The open standard behind it is **OpenPGP** (RFC 4880, updated by RFC 9580, verify). PGP uses the Web of Trust (3.20) for key trust. Early PGP used IDEA and RSA, later versions CAST-128 and AES.

Tools and commands used here. GPG (3.19), Mailvelope (3.22), Symantec Encryption (3.24).

Countermeasures and defense. Verify key fingerprints out of band. Use strong passphrases for private keys. Revoke lost keys with a revocation certificate.

EXAM FOCUS. "Hybrid cryptosystem", "random key encrypted with receiver's public key" -> PGP. "Compression, signing, encryption of emails and files" -> PGP.

> **TRAP:** The data is encrypted with the symmetric random key, NOT directly with RSA.

Memory trick. **P**GP = **P**ublic key wraps a **P**rivate session key.

3.19 GNU Privacy Guard (GPG)

Source: page 10, slide 22.

What it is. GPG (GnuPG) is a software replacement for PGP and a free implementation of the OpenPGP standard. It is also called hybrid encryption software because it uses both symmetric and asymmetric cryptography. The notes also say it supports S/MIME and Secure Shell (SSH).

Analogy. PGP is a name-brand tool. GPG is the free generic version that works the same way and speaks the same language.

How it works.

Diagram: GPG sign, encrypt, decrypt, verify

```
SENDER SIDE
 (a) raw file
 (b) sign with SENDER PRIVATE key
 (c) encrypt with RECEIVER PUBLIC key
 (d) send the signed and encrypted file by email or FTP

RECEIVER SIDE
 (a) decrypt with RECEIVER PRIVATE key
 (b) verify the signature with SENDER PUBLIC key
 (c) read the raw file
```

Real-world use. A Linux distribution signs its package files with GPG. A package manager verifies the signature before installing.

Tools and commands used here. `gpg` command (section 5.6).

Countermeasures and defense. Keep the private key offline or on a smart card, back up a revocation certificate, and verify fingerprints.

EXAM FOCUS. "Free implementation of OpenPGP" or "replacement for PGP" -> GPG. "Supports S/MIME and SSH" -> GPG per the notes.

> **TRAP:** GPG is not a different algorithm. It is software that follows OpenPGP, so it can talk to PGP.

Memory trick. **G**PG = **G**NU **G**ratis PGP (free).

3.20 Web of Trust (WoT)

Source: page 10, slide 23.

What it is. The Web of Trust is the trust model of PGP, OpenPGP, and GnuPG. There is no central CA. Everyone in the network acts as a CA and signs the keys of people they trust. WoT is a chain of individuals who validate each other's certificates with their signatures. Each user holds a **ring of public keys** and introduces other users they trust.

Person on slide	Role	Trust from Alice
Alice	Main user	Self
Alen, Bob, Smith	Friends	Direct trust
Henry, John	Outsiders	Indirect trust, through friends

[UNCLEAR on page 10] The slide's connecting lines were hard to follow. The roles above are the best reading: Alice trusts friends directly and outsiders only through them.

Diagram: Web of Trust (best reading)

```
Alice (main user)
  |-- direct trust ---> Alen (friend) ---- direct ---- Bob (friend)
  |-- direct trust ---> Smith (friend) --- indirect --> John (outsider)
  |-- indirect trust (through a friend) ---------------> Henry (outsider)
```

Analogy. Meeting a stranger because a friend vouches for them. You trust the stranger because someone you trust signed for them.

How it works.

1. Alice meets Bob, checks his identity, and signs his public key.
2. Bob signs Alen's key and Alice trusts Bob's judgment.
3. Alice now has indirect trust in Alen even without meeting.
4. More signatures on a key means more confidence in it.

Real-world use. Open-source developers hold key-signing parties to sign each other's keys in person.

> **ADDED CONTEXT:** PKI (3.13) is a hierarchy: a few trusted CAs sign everything. WoT is a mesh: any user can sign. WoT scales poorly for general public use, so the web uses PKI instead.

Countermeasures and defense. Verify fingerprints face to face or over a trusted channel, avoid signing keys you did not verify, and watch for fake keys.

EXAM FOCUS. "Everyone acts as a CA" or "ring of public keys" -> WoT. "Direct trust" and "indirect trust" -> WoT.

> **TRAP:** WoT belongs to PGP/OpenPGP/GnuPG. PKI with CAs belongs to SSL/TLS and S/MIME.

Memory trick. WoT = **W**ho-vouches-for-**T**hem.

3.21 Encrypting Email in Outlook: S/MIME Encryption

Source: page 11, slide 24.

What it is. Outlook can protect mail with **S/MIME** (Secure/ Multipurpose Internet Mail Extensions). S/MIME signs and encrypts mail using an X.509 certificate from a CA. Outlook needs a certificate for signing and one for encryption (often the same one).

How it works (the slide's steps).

1. Open File -> Options -> Trust Center -> Trust Center Settings.
2. Choose **Email Security** in the left pane.
3. In the **Encrypted email** section, click **Settings** beside Default Setting.
4. In the **Change Security Settings** window, go to **Certificates and Algorithms**.
5. Pick the S/MIME certificate for **Signing certificate** and **Encryption certificate**. Click OK.

The dialog on the slide shows these options: Encrypt contents and attachments for outgoing messages; Add digital signature to outgoing messages; Send clear text signed message when sending signed messages; Request S/MIME receipt for all S/MIME signed messages; Digital IDs (Certificates) with Publish to GAL (Global Address List); Hash Algorithm and Encryption Algorithm boxes; Send these certificates with signed messages; Read as Plain Text; and Script in Folders.

Analogy. Putting a tamper-evident seal (signature) and a lock (encryption) on every letter, using ID cards issued by a trusted office.

Real-world use. A law firm sends contracts by email. S/MIME proves the sender and hides the content.

> **ADDED CONTEXT:** To send encrypted mail you need the recipient's public certificate. You get it when they send you a signed message. S/MIME uses CA-issued certificates (hierarchical trust). PGP uses WoT.

AI angle. AI-written phishing and business email compromise look more convincing. A valid S/MIME signature proves the sender controls the certificate, which helps defenders separate real mail from impersonation. Example prompt: "Draft a staff notice explaining how to check for the signed-message ribbon in Outlook before acting on payment requests."

Countermeasures and defense. Enforce signing for sensitive mail, publish certificates to the GAL, revoke lost certificates, and keep private keys on smart cards or protected stores.

EXAM FOCUS. Menu path: Trust Center -> Email Security -> Encrypted email Settings -> choose S/MIME certificates. "Encrypt contents and attachments" is the main checkbox.

> **TRAP:** Signing and encrypting are separate options. Signing alone does not hide the message.

Memory trick. "**T**rust **C**enter -> **E**mail Security -> **S**ettings -> **C**ertificates" = "Tom Calls Every Shop Counter."

3.22 Email Encryption Tools

Source: page 11, slide 25.

What it is. Products that add encryption, signing, and tracking to email.

Tool	What it does	Type	Exam keyword
RMail (rmail.com)	Open tracking, delivery proof, encryption, e-signature, large files	Email add-on	Delivery proof
Mailvelope	Browser extension for OpenPGP in webmail	Extension	OpenPGP in browser
Virtru	End-to-end email and file protection	Add-on	Data protection
Webroot	Email encryption service (OpenText Webroot, verify)	Service	Email encryption
Secure Email (S/MIME) Cert (ssl.com)	Issues S/MIME certificates	CA service	S/MIME certificate
Proofpoint Email Protection	Gateway filtering and encryption	Gateway	Email security gateway

The RMail screenshot (RPost eSignOff service) shows an "E-SIGN RECORD: REGISTERED. SIGNED. TIME-STAMPED." message with date 03/15/2024, signer fields, and a message ID. [UNCLEAR on page 11] Most of the text is too small to read; names and IP address fields are blank.

Analogy. Registered mail with a signature on delivery, plus a sealed envelope.

Real-world use. A company sends a contract with RMail and later produces a registered receipt proving delivery and the exact time.

Countermeasures and defense. Combine gateway filtering (Proofpoint) with user-level encryption (S/MIME or PGP).

EXAM FOCUS. "Open tracking and delivery proof" -> RMail. "Browser extension for PGP" -> Mailvelope. "S/MIME certificates" -> SSL.com.

Memory trick. **R**Mail = **R**eceipt Mail (proof of delivery).

3.23 Disk Encryption

Source: page 12, slide 26.

What it is. Disk encryption protects the confidentiality of stored data by converting it into unreadable code, using software or hardware. It works like text-message

encryption, and it protects data even when the operating system is not running (for example, when a thief removes the drive). The slide's picture groups the idea into privacy, confidentiality, encryption, and protection, and shows passphrases, hidden volumes, volume encryption, USB flash drives, external hard drives, and backups as protected items.

Analogy. A safe for your hard drive. Without the combination (the passphrase or key) a thief who takes the safe still cannot open it.

How it works.

1. You choose what to protect: a whole disk, a partition, a container file (a **volume**), a USB drive, or a backup.
2. You set a passphrase or key (or use a TPM).
3. The software encrypts data "on the fly": data is encrypted as it is written and decrypted as it is read.
4. A **hidden volume** is a second encrypted volume placed inside free space of another one, so you can give up the outer password under pressure and keep the hidden data secret (ADDED CONTEXT).

> **ADDED CONTEXT:** Levels of protection. Full-disk encryption (FDE) covers the entire disk, including the OS, and asks for a password before boot (pre-boot authentication). Volume or container encryption protects one virtual disk file. File-level encryption protects single files. Disk encryption protects data at rest only. Once the disk is unlocked and the OS is running, malware can read it.

Real-world use. A sales laptop with BitLocker is stolen at an airport. The thief cannot read customer files from the drive.

Tools and commands used here. See 3.24.

Countermeasures and defense. Encrypt laptops, USB drives, and backups. Use strong passphrases or TPM plus PIN. Keep recovery keys in a secure vault. Shut down or hibernate (do not just sleep) when traveling, since keys can stay in RAM.

EXAM FOCUS. "Protects data when the OS is not active" or "data at rest" -> disk encryption. "Hidden volumes" -> VeraCrypt/Rohos style.

> **TRAP:** Disk encryption does not protect data in transit and does not stop an attack on a running, unlocked system.

Memory trick. **D**isk = **D**ata at rest.

3.24 Disk Encryption Tools

Source: page 12, slide 27.

What it is. Software that encrypts disks, volumes, and drives.

Tool	What it does (from the notes)	Platform	Exam keyword
VeraCrypt (veracrypt.fr)	Creates and maintains an on-the-fly encrypted volume	Windows, macOS, Linux	On-the-fly encrypted volume
Rohos Disk Encryption (rohos.com)	Hidden, encrypted partitions on PC, USB, or cloud	Windows	Hidden partition, cloud
FileVault (apple.com)	XTS-AES-128 with a 256-bit key on the startup disk	macOS	Apple startup disk
BitLocker Drive Encryption	Microsoft full-disk encryption	Windows	TPM plus recovery key
Symantec Encryption (Broadcom)	Enterprise disk and email encryption	Windows, macOS	Enterprise PGP lineage
SafeGuard Enterprise Encryption (Sophos)	Enterprise endpoint disk encryption	Windows, macOS	Sophos
Cryptsetup	Linux disk encryption front end (dm-crypt, LUKS)	Linux	Linux LUKS
Cryptmount	Lets users mount encrypted file systems	Linux	Linux user mounts

The VeraCrypt screenshot shows the main window with menus Volumes, System, Favorites, Tools, Settings, Help, Homepage; columns Drive, Volume, Size, Encryption Algorithm, Type; a sample volume `C:\Users\Admin\Desktop\M-Volume`; and buttons Create Volume, Volume Properties, Wipe Cache, Select File, Select Device, Mount, Auto-Mount Devices, Dismount All, and Exit. The option "Never save history" is also visible.

The Rohos screenshot shows an encrypted disk K: (Size 897 MB, Free 801 MB), with actions Encrypt Folder (choose OneDrive, Google, Dropbox), Create Backup, Encrypt USB drive (protected partition), Options (disk auto start, disconnect hotkeys), and Setup authentication key (hardware key). A "30-day trial" label is visible.

The FileVault screenshot shows Privacy & Security -> FileVault -> Turn On, and asks for an administrator's name and password.

> **ADDED CONTEXT: XTS** is a block cipher mode made for disk storage. XTS-AES-128 uses two AES-128 keys, so the stored key material is 256 bits. That is why FileVault is described as "XTS-AES-128 with a 256-bit key", not a contradiction. VeraCrypt is the successor of TrueCrypt and supports AES, Serpent, Twofish, Camellia, and cascades such as AES-Twofish-Serpent. LUKS (Linux Unified Key Setup) is the standard Linux disk encryption format.

Analogy. Different brands of safe for different homes: BitLocker for Windows houses, FileVault for Apple houses, LUKS for Linux houses, VeraCrypt for any house.

Real-world use. A consultant keeps client files in a VeraCrypt container on a USB stick. If the stick is lost, the container is unreadable without the password.

AI angle. None in the notes. As an ADDED CONTEXT point, keep AI model files and training data on encrypted volumes, since leaked model weights or datasets are valuable.

Countermeasures and defense. Keep recovery keys offline. Use strong passphrases (see the VeraCrypt brute-force attack in 3.30).

EXAM FOCUS. "On-the-fly encrypted volume" -> VeraCrypt. "XTS-AES-128 with 256-bit key" -> FileVault. "Hidden and encrypted partitions on cloud storage" -> Rohos. "Microsoft" -> BitLocker. "Sophos" -> SafeGuard. "Linux" -> Cryptsetup or Cryptmount.

Memory trick. Vera = **V**olume. **F**ile**V**ault = **F**ruit company (Apple). **B**it**L**ocker = **B**ill's **L**ocker (Microsoft).

3.25 Blockchain

Source: page 13, slide (unnumbered).

What it is. A **blockchain**, also called **distributed ledger technology (DLT)**, records and stores the history of transactions in **blocks**. For many transactions, many blocks are created and linked together, forming a blockchain. Each block contains a hash of the previous block, so changing an old block breaks every later link.

Process of creating a blockchain (5 steps).

1. The participant requests a transaction.
2. A block is created and broadcast to the members of the network.
3. Members validate the transaction.
4. The block is added to the blockchain.
5. A copy of the shared ledger is generated and made available to all members.

Types of blockchain.

Type	Who can join (ADDED CONTEXT)	Example idea
Public	Anyone	Bitcoin, Ethereum
Private	One organization controls access	Internal audit ledger
Federated	A fixed group of organizations	Bank consortium
Hybrid	Mix of public and private parts	Public proof, private data

Diagram: blocks linked by hashes

```
[Block A]        [Block B]        [Block C]
data             data             data
own hash   <---  prev hash        prev hash
                 own hash   <---  own hash
```

Analogy. A shared notebook that everyone has a copy of. Every page quotes the fingerprint of the page before. If someone rewrites page 3, the fingerprint on page 4 no longer matches and everyone notices.

> **ADDED CONTEXT:** Blockchains use hashes (SHA-256 in Bitcoin), digital signatures (ECDSA) to authorize transactions, and a **consensus mechanism** to agree on the next block. **Proof of Work (PoW)**: miners spend computing power (hash rate). **Proof of Stake (PoS)**: validators lock up coins (staking power). The attacks in 3.37 to 3.41 target these mechanisms.

Real-world use. Cryptocurrencies, supply-chain tracking, and tamper-evident record keeping.

Countermeasures and defense. Strong consensus rules, many independent nodes, waiting for several confirmations, audited smart contracts.

EXAM FOCUS. The 5 creation steps in order, and the 4 types (public, private, federated, hybrid).

Memory trick. Steps: **R**equest, **C**reate, **V**alidate, **A**dd, **S**hare ("Real Cats Vanish After Sunset"). Types: **Pu**blic, **Pr**ivate, **Fe**derated, **Hy**brid.

3.26 Cryptanalysis Methods

Source: pages 13 to 15, slides (unnumbered). Pages 13, 14, and 15 show the same slide being built up. The final version on page 15 includes the formulas.

What it is. Cryptanalysis is the study of breaking ciphers without knowing the key (or finding the key faster than brute force). The notes give four methods.

Method	Attacker knows	Works on	Key idea
Linear	Known plaintext	Block ciphers	Linear approximation leaks key bits
Differential	Chosen plaintext (originally)	Symmetric key algorithms	Study input differences vs output differences
Integral	Chosen plaintext sets	Substitution-permutation networks	Hold some bits fixed, run others through all values

Method	Attacker knows	Works on	Key idea
Quantum	Encrypted content plus quantum computer	Algorithms in general	Use quantum algorithms to speed up the break

Linear cryptanalysis.

- Commonly used on block ciphers. It is a **known-plaintext attack** and uses a **linear approximation** to describe the cipher's behavior.
- With enough pairs of plaintext and matching ciphertext, bits of information about the key can be obtained.
- Example in the notes: with a 56-bit DES key, brute force could take up to 2^56 attempts.
- Slide formula: `P_i XOR C_j = K_k`, where P_i is a bit from the plaintext, C_j a bit from the ciphertext, K_k a bit from the key, and XOR is the exclusive-or operation. In words: a chosen XOR of some plaintext and ciphertext bits equals a XOR of some key bits more often than chance. Collecting many pairs reveals which is likely.

Differential cryptanalysis.

- Applies to symmetric key algorithms. It examines how differences in the input affect differences in the output.
- It originally worked only with chosen plaintext. It can now also work with known plaintext and with ciphertext only.
- Slide formula: `dP -> dC`, where dP = P1 XOR P2 (difference between two plaintexts), dC = C1 XOR C2 (difference between the matching ciphertexts), and the arrow means the relationship inferred through analysis.

Integral cryptanalysis.

- Useful against block ciphers based on **substitution-permutation networks**. It is an extension of differential cryptanalysis.
- For block size b, it holds b-k bits constant and runs the other k through all 2^k possibilities. With k = 1 it is just differential cryptanalysis. With k > 1 it is a new technique.
- Slide formula: the XOR of the j-th bit of 16 ciphertexts equals 0 (`XOR for i=1 to 16 of C_i[j] = 0`). The set of 16 chosen plaintexts runs through all values of one nibble, so some output bit positions always XOR to zero. When a wrong key guess breaks this pattern, the attacker rejects the guess.

Quantum cryptanalysis.

- Cracking cryptographic algorithms with a quantum computer. The attacker must obtain the encrypted content. The process needs large time and quantum resources: **circuit width, circuit depth, number of gates, number of T-gates, T-depth, and MAXDEPTH**.

- Slide formula: **Grover's algorithm**, O(sqrt(N)), where N is the number of possible keys. It cuts a key search from O(N) to O(sqrt(N)).

> **ADDED CONTEXT:** Circuit width = number of qubits. Circuit depth = number of sequential steps. T-gates are the costly gates that make quantum circuits hard to build. MAXDEPTH is NIST's cap on circuit depth (verify). Linear cryptanalysis (Matsui, 1993) needs about 2^43 known plaintexts on DES. Differential cryptanalysis (Biham and Shamir) needs about 2^47 chosen plaintexts (verify both). **Shor's algorithm** breaks RSA, DH, DSA, and ECC outright. Grover halves effective symmetric key strength, so AES-256 stays at about 128-bit strength, which is why larger keys are the fix (3.45).

Analogy. Linear: you notice that a lock's spring tends to match the key's cuts. Differential: you test two nearly identical keys and watch how the lock reacts. Integral: you try a whole row of keys and check if the total cancels. Quantum: you hire a machine that can test many keys at once.

Real-world use. Researchers use these methods to show a new cipher is weaker than claimed, which led to the design of AES.

Tools and commands used here. CrypTool, CryptoSMT (3.43).

Countermeasures and defense. Use modern, public, well-analysed ciphers (AES). More rounds and good S-boxes resist linear and differential attacks. Use quantum-resistant algorithms and larger keys.

EXAM FOCUS. "Known plaintext + linear approximation" -> linear. "Differences between two plaintexts" -> differential. "Substitution- permutation networks, holds b-k bits constant" -> integral. "Quantum computer, Grover's, O(sqrt N)" -> quantum.

> **TRAP:** Linear = KNOWN plaintext. Differential = CHOSEN plaintext (originally). Do not swap them.

Memory trick. **L**inear = **L**ooks at known pairs. **D**ifferential = **D**ifferences of two. **I**ntegral = **I**ntegrates (sums) a set. **Q**uantum = **Q**ubits.

3.27 Cryptography Attacks

Source: pages 15 and 16, slides 31 and 32.

What it is. Attacks assume the cryptanalyst has access to the encrypted information. They differ in what else the attacker has or controls.

Attack	What the attacker has or does (from the notes)
Ciphertext-only	Has only ciphertext. Goal: recover the key from it.

Attack	What the attacker has or does (from the notes)
Adaptive chosen-plaintext	Sends a series of interactive queries, choosing each plaintext from earlier results.
Chosen-plaintext	Defines own plaintext, feeds it to the cipher, analyzes the ciphertext.
Related-key	Gets ciphertexts under two different keys. Useful if plaintext and matching ciphertext are known.
Dictionary	Builds a dictionary of plaintext with the matching ciphertext learned over time.
Known-plaintext	Knows part of the plaintext. Deduces the key to decrypt other messages.
Chosen-ciphertext	Gets plaintexts for an arbitrary set of ciphertexts of own choosing.
Rubber hose	Extracts secrets (such as a password) from a person by coercion or torture.
Chosen-key	Usually breaks an n-bit key cipher in 2^(n/2) operations.
Timing	Repeatedly measures the exact execution time of modular exponentiation.
Man-in-the-middle	Targets public-key systems where a key exchange happens before communication.

Analogy. Imagine a vault. Ciphertext-only: you see only the closed door. Known-plaintext: you also saw one thing put inside. Chosen- plaintext: you can ask the clerk to put any item inside and see the sealed result. Chosen-ciphertext: you can ask the clerk to open some sealed boxes you pick. Rubber hose: you threaten the owner.

How it works (ordering by attacker power, ADDED CONTEXT).

1. Ciphertext-only (weakest knowledge).
2. Known-plaintext.
3. Chosen-plaintext, then adaptive chosen-plaintext.
4. Chosen-ciphertext (strongest control).

Man-in-the-middle on key exchange (ADDED CONTEXT): the attacker sits between two parties, swaps in their own public key for each side, and reads or edits traffic while both sides think they talk directly. Authenticating public keys (certificates, signatures) stops it.

Timing attack: an RSA decryption takes slightly different times for different key bits. Measuring many operations reveals the key (Kocher, 1996, ADDED CONTEXT). Fix: constant-time code and blinding.

Real-world use. WEP's RC4 was broken by collecting many known- structure packets (known-plaintext style). Border agents or criminals may use rubber-hose pressure instead of any math.

Countermeasures and defense. Strong modern ciphers resist these on the math side. Add key authentication against MITM, constant-time code against timing, and policy and training against coercion and bribery.

EXAM FOCUS. The question describes what the attacker controls. "Only ciphertext" -> ciphertext-only. "Chooses plaintexts, sees ciphertexts" -> chosen-plaintext. "Chooses ciphertexts, gets plaintexts" -> chosen- ciphertext. "Torture or coercion" -> rubber hose. "Modular exponentiation time" -> timing.

> **TRAP:** The slide's "dictionary attack" here is a table of known plaintext and ciphertext pairs, not a password guessing list. Password cracking by wordlist is shown later in 3.30. Also the "chosen-key attack" text (2^(n/2)) is odd compared with textbooks; repeat the slide's wording if asked.

Memory trick. Order of knowledge: "**C**old **K**ids **C**hoose **A**ctive **C**oats" = Ciphertext-only, Known, Chosen-plaintext, Adaptive, Chosen-ciphertext.

3.28 Code Breaking Methodologies

Source: page 16, slide 33.

What it is. Ways to measure the strength of an encryption algorithm by trying to break it.

Method	What it is (from the notes)
Brute force	Discover keys by trying every possible combination
Frequency analysis	Study how often letters or groups of letters appear in ciphertext
Bribery and deceit	Use social engineering to extract cryptographic keys
One-time pad	Many non-repeating random groups of letters or number keys

Frequency analysis. In any stretch of written language, certain letters and letter groups occur at different rates. In English, E, T, A, O are very common. If one ciphertext letter appears most, it probably stands for E. This breaks simple substitution ciphers but not modern ciphers.

One-time pad (OTP). The key is random, as long as the message, used only once, and kept secret. The notes list it among the code-breaking methodologies, but it is

mainly an unbreakable cipher (ADDED CONTEXT). If the key is reused, the "many-time pad" attack becomes possible (tool: MTP, 3.43).

Analogy. Brute force: trying every number on a padlock. Frequency analysis: guessing a word puzzle by noticing which letters show up most. Bribery: paying the guard for the combination. OTP: a pad of one-use passwords that you tear off and destroy.

Real-world use. Spies used one-time pads for secret messages. Pen testers test social engineering (bribery and deceit) by calling staff.

Countermeasures and defense. Long random keys against brute force, modern ciphers against frequency analysis, staff training and key management against bribery and deceit, true OTP only if the key logistics can be handled.

EXAM FOCUS. "Letters appear with varying frequencies" -> frequency analysis. "Social engineering to get keys" -> bribery and deceit. "Non- repeating random key groups" -> one-time pad.

> **TRAP:** OTP is secure only if the key is truly random, as long as the message, and never reused.

Memory trick. **B**rute **F**orce, **F**requency, **B**ribery, **O**TP: "**B**ig **F**rogs **B**ite **O**ysters".

3.29 Brute-Force Attack

Source: page 17, slide 34.

What it is. Defeating a cryptographic scheme by trying a large number of possible keys until the correct one is found. It is a high-resource, time-intensive process, but it is more guaranteed to give results than other methods. Success depends on key length, time limits, and the system's security mechanisms.

Estimated time for a successful brute-force attack (slide table).

Power or cost	40 bits (5 char)	56 bits (7 char)	64 bits (8 char)	128 bits (16 char)
$2K (1 PC; an individual)	1.4 min	73 days	50 years	10^20 years
$100K (a company)	2 sec	35 hours	1 year	10^19 years
$1M (huge organization or state)	0.2 sec	3.5 hours	37 days	10^18 years

How to read it. Each extra key bit doubles the work. 56 bits to 64 bits is 8 more bits, so 2^8 = 256 times more work (73 days x 256 is about 51 years, matching the table). 128-bit keys are out of reach for any budget in the table.

Analogy. A padlock with 4 digits has 10,000 combinations. Add one digit and it has 100,000. Computers just spin the dial faster.

How it works.

1. Pick a candidate key.
2. Try to decrypt (or check against a hash).
3. If the result is not meaningful, try the next key.
4. Repeat until a key works.

Real-world use. A tester recovers a weak 40-bit export-grade key in minutes. A forensic examiner brute-forces a four-digit PIN on an old device.

Tools and commands used here. hashcat, John the Ripper (3.30).

Countermeasures and defense. Use keys of 128 bits or more (256 for symmetric when possible), long passphrases, slow key stretching (3.46), and rate limiting or lockout for online guessing.

EXAM FOCUS. "Tries every possible key" -> brute force. "Depends on key length" -> brute force. "Guaranteed but slow" -> brute force.

> **TRAP:** Brute force always works in theory, but not in practice if the key is long. Do not confuse it with a smart shortcut like cryptanalysis.

Memory trick. +1 bit = x2 time. Table anchors: 56 bits = DES = days to hours. 128 bits = forever.

3.30 Brute-Forcing VeraCrypt Encryption

Source: page 17, slide 35.

What it is. Attackers try to decrypt a VeraCrypt volume by guessing its password. They use the `dd` command to extract the hash value from the encrypted container, and then use hashcat or John the Ripper to brute-force the password against that hash.

Analogy. A safe's lock plate is hidden inside the safe's first page. The thief photocopies that one page (dd) and tries combinations at home on the copy (hashcat), without touching the safe again.

How it works (the slide's steps).

Diagram: VeraCrypt brute-force workflow

```
encrypted container
       |
       |  dd (copy the first 512 bytes)
       v
hash file (.tc)
       |
       |  hashcat -m 13721  (mask or wordlist)
       v
recovered password --> mount the volume
```

1. Extract the first 512 bytes (the volume header, which holds the salt and the encrypted header) from the container into a small file: `dd.exe if=<path_to_container> of=<path_to_hashfile.tc> bs=512 count=1`
2. Brute-force a numeric password (mask attack): `hashcat.exe -a 3 -w 1 -m 13721 <path_to_hashfile.tc> ?d?d?d?d`
3. Or try a wordlist of default or common passwords: `hashcat.exe -w 1 -m 13721 hash.tc wordlist.txt`

The screenshot shows `dd.exe if=E:\encrypted\target of=E:\encrypted\target_hash.tc bs=512 count=1`, the result lines "1+0 records in / 1+0 records out", the file `target` (131,072 KB) and the new `target_hash.tc` (1 KB shown in Explorer, 512 bytes exactly). A John the Ripper window is also shown. [UNCLEAR on page 17] The John window text reads "john version 0.4beta4" in the screenshot, which does not match John's usual version numbering. Best reading only.

Full flag-by-flag explanation is in sections 5.2 to 5.4.

Real-world use. A forensic examiner with legal authority recovers a weak four-digit password from a seized VeraCrypt container.

Tools and commands used here. dd, hashcat, John the Ripper, VeraCrypt.

AI angle. AI can build smarter guess lists. Example prompt for an authorized recovery: "From these facts about the owner (pet, year, city), suggest 50 password patterns and write the hashcat mask or rule strings that cover them." Defenders use the same idea to test whether users' passwords are guessable from public information.

Countermeasures and defense. Use a long random passphrase (not digits only), add keyfiles, raise the PIM (iteration multiplier), use hidden volumes, and never name the container file in a way that reveals its content. The high iteration count in VeraCrypt makes each guess slow.

EXAM FOCUS. The tools are `dd` (extract) and `hashcat` or `John the Ripper` (crack). `-m 13721` is the VeraCrypt SHA-512 mode. `-a 3` is brute-force (mask) attack mode. `?d` is a digit.

> **TRAP:** `dd` does not crack anything. It only copies the header. hashcat 13721 is labelled "legacy" in newer releases, which use modes 29421 and similar (verify with `hashcat --help`).

Memory trick. DD **D**igs out the header, **H**ashcat **H**ammers the password.

3.31 Meet-in-the-Middle Attack on Digital Signature Schemes

Source: page 18, slide 36.

What it is. The attack works by encrypting from one end and decrypting from the other, so the two meet in the middle. It can be used to forge messages that use multiple encryption schemes.

The slide's picture: the plaintext "John" is encrypted with key1 (1st, 2nd, ..., 256th) to give intermediate ciphertexts. The known ciphertext "AvBr" is decrypted with key2 (1st, 2nd, ..., 256th) to give intermediate values. The attacker looks for a match. [UNCLEAR on page 18] The exact picture labels are small. The above is the best reading of a toy example with 256 keys.

Analogy. Digging a tunnel from both sides of a mountain. Instead of one team digging the whole way (very slow), two teams dig half each and meet.

How it works.

1. The target uses double encryption: C = E_key2(E_key1(P)). The attacker knows one plaintext P and its ciphertext C.
2. For every possible key1, encrypt P and store the intermediate result.
3. For every possible key2, decrypt C and compare with the stored list.
4. A match gives the key pair. Work is about 2 x 2^n, not 2^(2n).

> **ADDED CONTEXT:** This is why Double DES (two 56-bit keys) offers only about 57 bits of strength, and why Triple DES is used, with an effective strength of 112 bits. The cost is memory to store the table.

Real-world use. Cryptographers use it to show that stacking two ciphers with independent keys does not double the strength.

Countermeasures and defense. Use one strong cipher with a long key (AES-256) rather than stacking weak ones. Do not design homemade cascades.

EXAM FOCUS. "Encrypts from one end, decrypts from the other" or "used against multiple encryption" -> meet-in-the-middle.

> **TRAP:** Do not confuse with man-in-the-middle (an active network interception). Meet-in-the-middle is a mathematical key search.

Memory trick. Meet-in-the-**M**iddle = **M**ath, two ends. **M**an- in-the-**M**iddle = **M**achine path (network).

3.32 Side-Channel Attack

Source: page 18, slide 37.

What it is. A side-channel attack is a physical attack on a cryptographic device or system to gain sensitive information. The cryptography runs on physical hardware (semiconductors, transistors, resistors) that is affected by power consumption, electromagnetic field, light emission, timing and delay, and sound. The attacker monitors these channels and uses the information for cryptanalysis.

Analogy. Guessing a safe's combination by listening to the clicks, or guessing a PIN by watching how long a clerk takes to type each digit. You never break the math. You use leaked "side effects".

How it works (the slide's model).

Diagram: main channel vs side channels

```
Input data --> [ Cryptographic operation ] --> Output data
(ciphertext)      (decryption inside a          (plaintext)
                   trusted security boundary)
                         |
             unintended "side" channels leak
                         |
    timing  power dissipation  electromagnetic fields  light
                         |
                         v
                Side-Channel Analysis
```

1. Data is decrypted inside a trusted zone and shown as plain text.
2. While it decrypts, physical factors such as timing and power dissipation change.
3. The attacker records these.
4. The attacker analyzes them to infer the key.

> **ADDED CONTEXT:** Variants: **timing** attack, **power analysis** (simple SPA, differential DPA), **electromagnetic** analysis, **acoustic** (sound), **optical** (light), and cache attacks. Fault injection is an active relative: the attacker causes errors on purpose.

Real-world use. Researchers extract an AES key from a smart card by measuring its power draw during many encryptions.

AI angle. Deep learning models are now used to analyze power or electromagnetic traces and find keys with fewer samples (ADDED CONTEXT). Defenders can use machine learning to test devices for leakage.

Countermeasures and defense (ADDED CONTEXT). Constant-time code, random delays, masking and blinding, noise generators, shielding, power filters, tamper-resistant hardware, and limiting physical access.

EXAM FOCUS. "Physical attack, power consumption, electromagnetic, timing, sound" -> side-channel.

> **TRAP:** A side-channel attack does not break the algorithm. It reads what the device leaks while running it.

Memory trick. **S**ide-channel = **S**ounds, **S**ignals, **S**ecs (time), **S**ource of power.

3.33 Hash Collision Attack

Source: page 19, slide 38.

What it is. A hash collision attack finds two different input messages that produce the same hash output. This lets an attacker abuse digital signatures: sign one message, then swap in another with the same hash. The notes use SHA-1 as the example: it converts input into a constant-length string that acts as a fingerprint of the file. The attacker can forge the victim's digital signature of message a1 on the wrong message a2. Once one collision is found, the attacker tries to find more by concatenating data to the matching messages.

Analogy. Two different people whose fingerprints are identical. If a guard only checks the fingerprint, either person gets in.

How it works.

1. The attacker crafts two messages, a1 (harmless) and a2 (malicious), with the same hash.
2. The victim signs a1. The signature covers the hash.
3. The attacker attaches that signature to a2.
4. The verifier hashes a2, finds the hash equals the signed one, and accepts a2 as genuine.

> **ADDED CONTEXT:** The **birthday attack** finds a collision after about 2^(n/2) tries for an n-bit hash (for MD5, about 2^64). Real cases: MD5 collisions allowed a forged CA certificate (2008) and the Flame malware abused one (2012). Google and CWI published the first SHA-1 collision (SHAttered) in 2017.

Real-world use. A researcher shows two PDF files with different content and the same SHA-1 hash.

Countermeasures and defense. Stop using MD5 and SHA-1 for signatures and certificates. Use SHA-256, SHA-3, or stronger. Hash lengths of 256 bits or more (slide 52 item 9).

EXAM FOCUS. "Two different inputs, same hash" -> collision. "Forge a signature of a1 onto a2" -> hash collision attack.

> **TRAP:** A collision is not the same as breaking one-wayness. The attacker does not recover an original input from a hash.

Memory trick. Collision = **C**opy-cat hashes (twins).

3.34 DUHK Attack

Source: page 19, slide 39.

What it is. DUHK stands for **Don't Use Hard-Coded Keys**. It is a vulnerability that lets an attacker obtain the encryption keys used to secure VPNs and web sessions.

1. It mainly affects hardware or software that uses the **ANSI X9.31** random number generator (RNG) with a hard-coded seed key.
2. A **pseudorandom number generator (PRNG)** creates random-looking bit sequences from a secret starting value, the **seed**, and its current state.
3. The two key problems are the X9.31 RNG and the hard-coded seed. An attacker who combines X9.31 with the hard-coded seed key can decrypt data sent or received by that device.
4. Attackers identify encryption keys and steal critical business data, user credentials, and credit card details.

Analogy. A lottery machine that always starts with the same secret number. Once you know it, you can predict every ball.

How it works.

1. The device uses X9.31 with a seed key written in firmware (the same in every unit).
2. The attacker extracts the seed key from the firmware.
3. The attacker watches some of the device's raw output in a handshake.
4. This reveals the RNG state, so all future (and some past) random values, and the session keys built from them, can be computed.

> **ADDED CONTEXT:** Disclosed in 2017. Fortinet FortiOS was a named example (verify). X9.31 is retired by NIST.

Real-world use. A firewall appliance ships with a hard-coded seed. A passive listener on the network recovers VPN keys and reads traffic.

Countermeasures and defense. Update firmware, use approved RNGs (for example NIST SP 800-90A DRBGs, ADDED CONTEXT), never hard-code keys or seeds, and use perfect forward secrecy so one leak does not expose past sessions.

EXAM FOCUS. "Hard-coded keys", "ANSI X9.31 RNG" -> DUHK.

> **TRAP:** DUHK is about a predictable random number generator, not about weak encryption algorithms.

Memory trick. DUHK = "**D**uh, **U**se no **H**ard-coded **K**eys".

3.35 DROWN Attack

Source: page 20, slide 40.

What it is. A DROWN attack is a cross-protocol weakness. It targets servers that support modern SSLv3/TLS protocol suites but also accept the obsolete SSLv2 with the same private key. It affects HTTPS and other services that depend on SSL and TLS. The attacker decrypts a recent TLS connection between a victim client and the server by sending malicious SSLv2 probes that use the same private key. It is performed as part of an online MitM attack to break the keys and sniff secrets such as passwords and bank account details.

> **ADDED CONTEXT:** DROWN = Decrypting RSA with Obsolete and Weakened eNcryption (2016, CVE-2016-0800, verify). It abuses an RSA padding oracle in SSLv2.

Analogy. A bank has a new steel front door (TLS) and an old wooden back door (SSLv2), but both open with the same key. A thief tests the back door until the lock gives away the key.

How it works (the slide's diagram).

Diagram: DROWN

```
Victim client <===== TLS connection =====> Victim server (also SSLv2)
       ^                                          ^
       |                                          |
attacker captures the TLS traffic     attacker sends malicious
                                      SSLv2 probes (same private key)
```

1. The attacker records a TLS handshake or session.
2. The attacker sends many SSLv2 probes to a server that shares the RSA key.
3. The server's answers leak information.
4. The attacker recovers the session key and decrypts the TLS traffic.

Real-world use. About a third of HTTPS servers were exposed when the attack was announced (verify). A mail server with SSLv2 enabled using the same certificate as the web server was enough.

Countermeasures and defense. Disable SSLv2 on every server and service (web, mail, VPN). Do not reuse the private key on servers that allow SSLv2. Update OpenSSL. Test with an online DROWN checker or Nmap (ADDED CONTEXT).

EXAM FOCUS. "SSLv2 probes", "cross-protocol", "same private key" -> DROWN.

> **TRAP:** DROWN exploits an old protocol to attack a new one. The attacked connection can use up-to-date TLS.

Memory trick. DROWN the old SSLv2 to save TLS.

3.36 Rainbow Table Attack

Source: page 20, slide 41.

What it is. A rainbow table attack reverses cryptographic hash functions with a precomputed table. The table holds word lists (such as dictionaries and brute-force lists) and their hash values. It uses the cryptanalytic **time-memory trade-off**: spend time once to build the table, then look up hashes quickly, which needs less time than some other techniques. The attacker compares hashes against the table; a match reveals the password.

> **ADDED CONTEXT:** The slide's last line says the attacker computes the hash for a list of passwords and compares it to the table. In practice the table stores compressed hash chains, and the attacker looks up the stolen hash in it. Either way the key idea is the same: precomputed hashes.

Analogy. A phone book printed in reverse: instead of computing each answer, you look up the number and find the name.

How it works.

1. Pre-compute hashes for a huge list of likely passwords and store them (as chains in a rainbow table).
2. Steal the target's password hashes.
3. Look each hash up in the table.
4. A match gives the original password.

Real-world use. A tester dumps NTLM or unsalted MD5 hashes and recovers many passwords in minutes.

Tools and commands used here (ADDED CONTEXT). RainbowCrack (`rtgen`, `rtsort`, `rcrack`), Ophcrack, Winrtgen.

Countermeasures and defense. Salt each password (random, unique value added before hashing) so a table made for unsalted hashes is useless. Use slow hashes (bcrypt, PBKDF2, scrypt, Argon2) and long passwords.

EXAM FOCUS. "Precomputed table of hashes", "time-memory trade-off" -> rainbow table. Best defense -> salting.

> **TRAP:** Salting does not stop brute force of one hash. It stops the reuse of precomputed tables.

Memory trick. Rainbow = **R**eady-made hashes. **S**alt **S**preads **S**ame passwords into different hashes.

3.37 Blockchain Attack: 51% Attack

Source: page 21, slide (unnumbered).

What it is. A 51% attack (also called a **majority attack**) happens when an attacker or group controls more than 50 percent of a blockchain's computational power (**hash rate**) or **staking power**. This control lets them manipulate the chain: double-spending, denial of service, transaction reversals, and other malicious activity.

Analogy. A vote where one side secretly owns 51 percent of the votes. They can overrule everyone and rewrite the minutes.

How it works (the slide's steps).

1. The attacker separates their mining pool from the blockchain network.
2. The attacker gains 51 percent or more of the total mining power.
3. The attacker secretly adds blocks to a private chain.
4. The attacker reconnects the private chain to the public network.
5. The shorter (honest) chain is discarded, and the attacker's longer chain wins, giving control over the network's history.

Real-world use. Smaller proof-of-work coins have suffered 51% attacks because renting enough hash power is cheap compared with Bitcoin (ADDED CONTEXT).

Countermeasures and defense (ADDED CONTEXT). Large, decentralized networks, more confirmations before accepting large payments, monitoring for chain reorganizations, checkpoints, and penalties (slashing) in proof of stake.

EXAM FOCUS. "More than 50 percent hash rate or staking power" -> 51% or majority attack. Enables double spending.

> **TRAP:** The attacker cannot steal coins from private wallets. They can reverse their own transactions and block others.

Memory trick. 51 = **fi**fty-**fi**ve-ish: "Fifty-one means my chain is the one."

3.38 Blockchain Attack: Finney Attack

Source: page 21, slide (unnumbered).

What it is. A Finney attack uses the delay between broadcasting a transaction and its confirmation to reverse it. The attacker double-spends cryptocurrency to get goods or services for free while keeping the coins.

Analogy. Writing a check, keeping a second copy of the money in a sealed envelope that you deposit a moment later, so the shop's check bounces.

How it works (the slide).

1. The attacker mines a block, but does not broadcast it. The block includes a transaction sending some coins to the attacker's own address. This is the **pre-mined block**.
2. The attacker starts a transaction with a victim (merchant), using the same coins, for goods or services.
3. The victim accepts the transaction (without waiting for confirmations) and sends the goods.
4. The attacker releases the pre-mined block. The network validates it.
5. The network invalidates the victim's transaction, because those coins are already spent.

Countermeasures and defense. Wait for confirmations before delivering goods (do not accept zero-confirmation payments for valuable items).

EXAM FOCUS. "Pre-mined block", "double spend", "before confirmation" -> Finney.

> **TRAP:** Finney needs pre-mining. A race attack (3.40) does not.

Memory trick. Finney = **F**irst-mine, **F**ake-pay, **F**ree goods.

3.39 Blockchain Attack: Eclipse Attack

Source: page 22, slide (unnumbered).

What it is. An Eclipse attack isolates a target node from the rest of the network by surrounding it with malicious nodes, so the attacker controls the node's view of the blockchain. Uses: disrupt transaction processing, split mining power, and enable double spending.

Analogy. Surrounding one person at a party so every conversation they hear comes from your friends.

How it works (the slide).

1. The attacker launches a DDoS attack on the target node.
2. The attacker fills the target's **peer table** with the IP addresses of malicious nodes.
3. The target node disconnects from legitimate nodes when it restarts.

4. The target checks its peer table for new connections.
5. The peer table returns malicious IPs, and the target connects only to the attacker's nodes.

Countermeasures and defense (ADDED CONTEXT). Increase the number of connections, choose peers randomly and from diverse networks, limit inbound connections, whitelist trusted peers, and protect nodes from DDoS.

EXAM FOCUS. "Peer table", "isolate a node", "malicious nodes" -> Eclipse.

> **TRAP:** Eclipse targets one node, while 51% targets the whole network's majority power.

Memory trick. Eclipse hides the sun: the node sees only the attacker.

3.40 Blockchain Attack: Race Attack

Source: page 22, slide (unnumbered).

What it is. A race attack is a double-spending attack that exploits delays in transaction confirmation to get goods or services without paying. Unlike the Finney attack, it needs no pre-mined block. It relies on the attacker's ability to broadcast transactions fast and use network latency.

How it works (the slide).

1. The attacker creates two transactions using the same coins.
2. Transaction A (to the victim) is broadcast to the victim and the network.
3. The attacker immediately broadcasts Transaction B (to themselves) to the network.
4. The network confirms Transaction B first. Transaction A is invalidated.
5. The victim already handed over the goods.

Analogy. Handing the cashier a check and, a second later, cashing a copy of the same check at the bank next door.

Countermeasures and defense. Wait for confirmations, do not accept unconfirmed payments, and use connections to well-connected nodes (ADDED CONTEXT).

EXAM FOCUS. "Two conflicting transactions, no pre-mining" -> race.

> **TRAP:** Finney = pre-mined block. Race = two transactions racing.

Memory trick. Race = two **R**unners, one coin.

3.41 Blockchain Attack: DeFi Sandwich Attack

Source: page 23, slide (unnumbered).

What it is. A DeFi sandwich attack abuses the time delay and order of execution on **decentralized exchanges (DEXs)** to move a token's price. It targets tokens with large pending trades. Victims buy at an inflated price or sell at a deflated price and lose money.

Analogy. Someone sees you about to buy all the apples at a stall, buys them first to push the price up, lets you buy at the high price, then sells the apples back for profit.

How it works (the slide).

Diagram: the sandwich

```
mempool order:  [attacker BUY] -> [victim BUY] -> [attacker SELL]
token price:    low               pushed up       attacker sells high
```

1. The attacker watches the **mempool** (pool of pending transactions) for a large pending trade.
2. The attacker places a **front-run** buy order for the same token.
3. The victim's trade executes at the higher price.
4. The attacker places a sell order for the token and profits from the inflated price.

Countermeasures and defense (ADDED CONTEXT). Set tight slippage limits, use private transaction relays, use exchanges with batch matching, and split large trades.

EXAM FOCUS. "Mempool", "front-run", "DEX" -> sandwich attack.

> **TRAP:** The victim's trade is the filling between attacker buy and sell.

Memory trick. Bread (attacker buy), meat (victim), bread (attacker sell).

3.42 Quantum Computing Attacks

Source: page 23, slide 48.

What it is. The notes list 13 attacks in the quantum computing era, by name only. The plain-language descriptions below are ADDED CONTEXT (verify against the official courseware).

#	Attack	Plain meaning
1	Quantum Cryptanalysis	Break RSA, ECC, and other keys with a quantum computer
2	Quantum Side-Channel	Exploit physical leaks of quantum devices
3	Classical-to-Quantum Transition	Abuse the migration period of mixed old and new crypto

#	Attack	Plain meaning
4	Harvest-Now, Decrypt-Later	Steal encrypted data now, decrypt it when quantum arrives
5	Quantum Trojan Horse	Probe quantum devices with injected light pulses
6	Quantum Supply Chain	Tamper with quantum hardware or software in transit
7	Quantum Computer Sabotage	Disrupt or damage a quantum computer's operation
8	Fault Injection on Quantum Hardware	Cause deliberate errors in qubits or gates
9	Quantum Denial-of-Service	Overload or jam quantum resources or channels
10	Quantum Data Eavesdropping	Intercept quantum channel data
11	Quantum Bit Flipping	Change qubit values to alter results
12	Quantum Error Correction Exploitation	Abuse the error-correction process
13	Quantum Replay	Resend captured quantum messages

Analogy. Harvest-now-decrypt-later: a burglar stealing a locked safe today because he knows a better lock-pick will exist in ten years.

How it works (the most tested one).

1. Today the attacker records and stores encrypted traffic (VPN, TLS, backups).
2. The data is safe against today's computers.
3. Later, a large quantum computer using Shor's algorithm breaks the RSA or ECC key exchange and decrypts the stored data.
4. Long-lived secrets (health, state, trade secrets) are the target.

Real-world use. Governments and banks now plan quantum-safe migration because data stolen today may stay valuable for 10 to 30 years.

Countermeasures and defense. See 3.45.

EXAM FOCUS. "Store now, decrypt later" -> harvest-now-decrypt-later. Count the list: 13 attacks.

> **TRAP:** Harvest-now-decrypt-later threatens data in transit and at rest TODAY, even though the quantum computer does not exist yet.

Memory trick. HN**D**L = "**H**oard **N**ow, **D**ecrypt **L**ater."

3.43 Cryptanalysis Tools

Source: page 24, slide 50.

What it is. Tools that help study, attack, or test cryptography.

Tool	What it does	Exam keyword
CrypTool (cryptool.org)	E-learning program for cryptography and cryptanalysis (CT1, CT2, JCT, CTO)	Learning and classic ciphers
RsaCtfTool	Recovers private keys from weak RSA public keys	RSA CTF attacks
Msieve	Factors large integers (for example RSA moduli)	Integer factoring
Cryptol	Language for specifying cryptographic algorithms (ADDED CONTEXT)	Crypto specification
CryptoSMT	Cryptanalysis of symmetric ciphers with SMT/SAT solvers	SMT solver attacks
MTP	Attacks messages encrypted with a reused one-time pad key	Many-time pad

The CrypTool screenshot shows CrypTool 1 with an Enigma rotor wiring diagram and the text `DASOBERKOMMANDODERWEHRMAQTIGTSTBEKANNTX`. [UNCLEAR on page 24] The text looks like a famous German wartime broadcast line with typos; best reading only.

> **ADDED CONTEXT:** CT1 is the classic Windows program. CT2 is the graphical workflow version. JCT (JCrypTool) is Java-based. CTO is CrypTool-Online in the browser. MTP and CryptoSMT were checked against their public project pages.

Analogy. A lab bench for code breakers.

Real-world use. Students use CrypTool to encrypt with the Enigma and break the Caesar cipher by frequency analysis. CTF players use RsaCtfTool when an RSA key is short.

Countermeasures and defense. Use long keys and standard algorithms so these tools fail.

EXAM FOCUS. "Cryptanalysis e-learning program" -> CrypTool. "Factor RSA modulus" -> Msieve. "SMT solvers" -> CryptoSMT.

Memory trick. Cryp Tool teaches. **Msieve** sieves numbers.

3.44 How to Defend Against Cryptographic Attacks

Source: page 24, slide 52.

What it is. Twelve countermeasures from the notes, each with a plain explanation.

1. **Give access to keys directly to the application or user.** Do not share keys through middlemen. Keep access tight.
2. **Deploy an IDS to monitor key exchange and access.** Alert on suspicious key use.
3. **Encrypt keys stored on disk with passphrases or passwords.** A stolen key file is then not enough.
4. **Do not put keys inside source code or binaries.** Hard-coded keys leak (see DUHK, 3.34).
5. **Do not transfer private keys for certificate signing.** The private key stays where it was created.
6. **Prefer 256-bit keys for symmetric algorithms**, especially for large transactions.
7. **Add message authentication to symmetric protocols** (HMAC or an authenticated mode such as GCM) so tampering is detected.
8. **Use asymmetric keys of at least 2048 bits** for secure applications.
9. **Use hash lengths of 256 bits or more.**
10. **Prefer recommended tools and products to self-engineered algorithms.** Never roll your own crypto.
11. **Avoid simple relationships between keys.** Create each key from a key derivation function (KDF).
12. **Prefer hash outputs with a higher bit length**, which makes reversing or guessing harder.

Analogy. Locking the house is not enough. You also hide the spare key properly, add an alarm, and do not build your own lock out of scrap metal.

Real-world use. A developer moves an API secret from code into a vault, and the team rotates it every 90 days (ADDED CONTEXT).

AI angle. AI coding assistants can suggest code with embedded keys or old algorithms. Secret scanners and review prompts catch it. Example prompt: "Scan this repository diff for hard-coded keys, tokens, and weak cryptographic calls, and list file and line."

EXAM FOCUS. Numbers to memorize: 256 (symmetric), 2048 (asymmetric), 256 (hash). Keys must not be in source code. Use KDF. Do not self-engineer algorithms.

> **TRAP:** "At least 2048 bits" applies to asymmetric keys, not to symmetric ones. AES-256 is not a 2048-bit key.

Memory trick. "**256 / 2048 / 256**": symmetric, asymmetric, hash.

3.45 How to Defend Against Quantum Computing Attacks

Source: page 25, slide 54.

What it is. Ten countermeasures from the notes.

1. **Use larger keys for symmetric cryptography.** Grover's algorithm halves their strength, so AES-256 stays strong.
2. **Add quantum-resistant digital signatures to blockchain protocols.**
3. **Encrypt stored data with quantum-resistant algorithms.**
4. **Split data into fragments across multiple locations**, so the original cannot be rebuilt.
5. **Develop quantum-specific firewalls** to filter and protect quantum communication channels.
6. **Use quantum-resistant zero-knowledge proofs** to authenticate users (prove knowledge of a secret without revealing it).
7. **Use quantum-resistant distributed ledger technology** for decentralized transaction records.
8. **Use TPMs that support quantum-resistant algorithms** for secure boot.
9. **Add quantum-resistance checks to the SDLC and code review.**
10. **Use HSMs for storing quantum-resistant keys**, and update them regularly.

> **ADDED CONTEXT:** Post-quantum cryptography (PQC) standards from NIST: ML-KEM (FIPS 203, key exchange), ML-DSA (FIPS 204, signatures), SLH-DSA (FIPS 205, hash-based signatures). FN-DSA (FIPS 206) was still a draft and HQC was chosen in 2025 as a backup key mechanism (verify current status). **Crypto-agility** means designing systems so algorithms can be swapped without a rebuild. A common migration step is **hybrid** key exchange (classical plus post-quantum).

Analogy. Replacing all locks in a building before a master lock-pick becomes available.

EXAM FOCUS. "Larger symmetric keys", "quantum-resistant signatures", "HSM", "TPM", and "fragment data" belong here.

> **TRAP:** Doubling symmetric keys (AES-128 to AES-256) is enough against Grover. It is NOT enough for RSA or ECC. Those need new algorithms.

Memory trick. Sym = bigger keys. Asym = new algorithms.

3.46 Key Stretching

Source: page 25, slide 55.

What it is. Key stretching strengthens a key that might be slightly too weak, usually by making it longer and costlier to guess. It turns a short password into a strong derived key by running slow, repeated computation, so each brute-force guess costs the attacker much more.

Function	From the notes	Plain meaning
PBKDF2	Part of PKCS #5 v2.01. Applies a function (such as hash or HMAC) to the password and a salt to produce a derived key	Repeat HMAC thousands of times with a salt
bcrypt	Used with passwords. Uses a derivation of the Blowfish algorithm, converted into a hashing algorithm, to hash a password and add a salt	Slow Blowfish-based hash with built-in salt

Analogy. Replacing a pencil-thin door chain with a long winding corridor of locked doors: the thief must pass every one for every guess.

How it works.

1. Take the password and a random salt.
2. Run the hash or HMAC thousands of times (iterations), feeding the output back in.
3. Store the salt, the iteration count, and the result.
4. When the user logs in, repeat the same work and compare.

> **ADDED CONTEXT:** bcrypt has a cost factor that sets the work. OWASP guidance for PBKDF2-HMAC-SHA256 is hundreds of thousands of iterations (verify the current number). Newer **scrypt** and **Argon2** add memory hardness to resist GPUs. A **salt** is a random value stored with the hash. It defeats rainbow tables. A **pepper** is a secret value kept outside the database.

Real-world use. A website stores bcrypt hashes. If the database leaks, attackers face a very slow guessing rate, compared with fast MD5.

Countermeasures and defense. Always salt, choose a cost that takes about 100 to 500 milliseconds per login, and raise it over time.

EXAM FOCUS. "Makes weak keys stronger by lengthening or slowing" -> key stretching. "PKCS #5" -> PBKDF2. "Blowfish-based password hash" -> bcrypt.

> **TRAP:** Key stretching slows each guess. It does not stop an attacker who guesses a very weak password on the first try.

Memory trick. PBKDF2 = **P**assword-**B**ased. **b**crypt = **B**lowfish **crypt**.

4. Tool Encyclopedia

Grouped by job. "Phase used" follows the five hacking phases where it applies best; for defensive or neutral tools it lists the main use case instead.

4.1 Hashing and Checksum Tools

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
HashMyFiles	Shows MD5, SHA-1, more for files	Windows	Integrity check	Local file hash
MD5 Calculator	Computes MD5 for files/text	Windows	Integrity check	MD5 only
All Hash Generator	Many hash types, one input	Web	Integrity check	Browser, many algorithms
Message Digester	Digests text, many algorithms	Web	Integrity check	Online digest
CyberChef	Chains hash/ encode/decode "recipes"	Web (GCHQ)	Analysis, CTF	Nested/multi-layer hashing
hashcat	GPU password/ hash cracker	CLI, cross-platform	Gaining access	Mask attack, -m mode
John the Ripper	CPU/GPU password cracker	CLI, cross-platform	Gaining access	Wordlist cracking
RainbowCrack	Builds and uses rainbow tables	CLI/GUI	Gaining access	Precomputed table lookup

4.2 Text, File, and Disk Encryption Tools

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
BCTextEncoder	Encrypts text into a pasteable block	Windows	Confidentiality	Encoded message block
CryptoForge	Encrypts files, folders, email	Windows	Confidentiality	File/folder suite
AxCrypt	Right-click AES file encryption	Windows, macOS	Confidentiality	Simple file encryption
Concealer	Encrypts files into vaults	macOS	Confidentiality	Mac file vault
SensiGuard	File encryption, secure delete	Windows	Confidentiality	File security

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
VeraCrypt	On-the-fly encrypted volumes	Win, macOS, Linux	Confidentiality	Encrypted container
Rohos Disk Encryption	Hidden partitions, cloud folders	Windows	Confidentiality	Hidden partition
FileVault	XTS-AES-128, 256-bit key startup disk	macOS	Confidentiality	Apple full-disk
BitLocker	Full-disk encryption with TPM	Windows	Confidentiality	Microsoft full-disk
Symantec Encryption	Enterprise disk/ email encryption	Win, macOS	Confidentiality	Enterprise PGP lineage
SafeGuard Enterprise	Endpoint disk encryption	Win, macOS	Confidentiality	Sophos
Cryptsetup	LUKS disk encryption front end	Linux	Confidentiality	Linux LUKS
Cryptmount	Mounts encrypted file systems	Linux	Confidentiality	Linux user mount

4.3 Email and PKI Tools

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
GPG (GnuPG)	Free OpenPGP implementation	CLI, cross-platform	Confidentiality, signing	Hybrid, S/MIME, SSH
Mailvelope	OpenPGP inside a webmail browser	Browser extension	Confidentiality	PGP in browser
RMail	Tracking, delivery proof, encryption	Email add-on	Nonrepudiation	Delivery proof
Virtru	End-to-end email/ file protection	Add-on	Confidentiality	Data protection
Proofpoint Email Protection	Email security gateway	Gateway	Defense	Gateway filtering
SSL.com S/MIME certs	Issues S/MIME certificates	CA service	PKI	S/MIME certificate
Java keytool	Creates self-signed certs/keys	CLI	PKI	Self-signed certificate

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
Apple Keychain	Stores and creates certificates	macOS	PKI	Self-signed certificate

4.4 Cryptanalysis and Research Tools

Tool	What it does (simple)	Type/Platform	Phase/Use	Exam keyword
CrypTool (CT1/ CT2/JCT/CTO)	E-learning for crypto/ cryptanalysis	Win, Java, Web	Learning, analysis	Enigma, classic ciphers
RsaCtfTool	Recovers private key from weak RSA	CLI (Python)	Gaining access	Weak RSA, CTF
Msieve	Factors large integers	CLI	Gaining access	Factor RSA modulus
Cryptol	Language to specify crypto algorithms	DSL/CLI	Verification	Spec and verify ciphers
CryptoSMT	SMT/SAT solver cryptanalysis	CLI (Python)	Research	Differential/linear search
MTP	Breaks reused one-time pad keys	CLI (Python)	Gaining access	Many-time pad
OpenSSL	Crypto toolkit; SSL/TLS, ciphers, hashes	CLI library	All phases	`openssl enc`, `s_client`

> **ADDED CONTEXT:** Close alternatives the exam often places next to these tools: **Hashcat vs John the Ripper** (both crack hashes; hashcat favors GPU/mask attacks, John favors CPU and format autodetection). **VeraCrypt vs BitLocker vs FileVault** (cross-platform container vs Windows-native vs macOS-native full-disk). **GPG vs OpenSSL** (GPG for OpenPGP email/file workflows, OpenSSL for TLS/certs and raw cipher operations). **CyberChef vs CrypTool** (CyberChef is a web "recipe" chainer for data munging; CrypTool is a dedicated teaching tool for classic and modern cryptanalysis).

5. Command Sheet (I have no lab, so teach me to read commands)

5.1 openssl enc -ciphers (and openssl enc for encryption)

Purpose. List the symmetric ciphers an OpenSSL build supports, or encrypt/decrypt a file with a chosen cipher. The slide's terminal (Parrot Linux) runs the list form.

Full syntax.

```
openssl enc -ciphers
openssl enc -<cipher> [-pbkdf2] [-salt] -pass pass:<password> \
    -in <infile> -out <outfile>
openssl enc -d -<cipher> [-pbkdf2] -pass pass:<password> \
    -in <infile> -out <outfile>
```

Breakdown of each part.

Part	Meaning
`enc`	OpenSSL's symmetric encryption/decryption sub-command
`-ciphers`	Print the list of supported cipher names and exit
`-<cipher>`	Cipher name, for example `-aes-256-cbc`
`-d`	Decrypt instead of encrypt
`-pbkdf2`	Use PBKDF2 to derive the key from the password (recommended)
`-salt`	Add a random salt (default in modern OpenSSL)
`-pass pass:<password>`	Supply the password on the command line
`-in <file>`	Input file
`-out <file>`	Output file

Flag table.

Flag	Meaning	When to use	Example
`-ciphers`	List supported ciphers	Checking what a server/build supports	`openssl enc -ciphers`
`-aes-256-cbc`	AES, 256-bit key, CBC mode	Encrypting a file at rest	`openssl enc -aes-256-cbc ...`
`-pbkdf2`	Strong key derivation from password	Always, for password-based encryption	`-pbkdf2`
`-d`	Decrypt	Reversing a previous encryption	`openssl enc -d -aes-256-cbc ...`

Flag	Meaning	When to use	Example
`-base64`	Output/accept base64 text	Sending ciphertext as text (email)	`-base64`

Demo commands in a realistic scenario. A tester receives a password from a colleague by phone and encrypts a findings file before emailing it.

```
openssl enc -ciphers
openssl enc -aes-256-cbc -pbkdf2 -salt \
    -pass pass:S3cretPhrase! -in report.txt -out report.enc
openssl enc -d -aes-256-cbc -pbkdf2 \
    -pass pass:S3cretPhrase! -in report.enc -out report_out.txt
```

Sample output (illustrative; this sandbox has no openssl binary, so treat the values as representative, not a captured run).

```
Supported ciphers:
-aes-128-cbc -aes-128-cfb -aes-128-cfb1 -aes-128-cfb8
-aes-128-ctr -aes-128-ecb -aes-128-ofb -aes-192-cbc
-aes-192-cfb -aes-256-cbc -aes-256-ctr -aes-256-ecb
-aria-128-cbc -bf -bf-cbc -bf-ofb -blowfish
```

Reading it: each name is `<algorithm>-<keysize>-<mode>`. `aes-256-cbc` is AES, 256-bit key, CBC mode. `bf` and `blowfish` are Blowfish aliases. Names without a mode suffix (`aes128`, `aes128-wrap`) are key-wrap variants used for wrapping other keys, not for bulk data.

Common mistakes. Forgetting `-pbkdf2` lets OpenSSL use a weaker, older key derivation. Forgetting `-d` on decrypt re-encrypts garbage. Reusing the same salt/IV for many files with the same password weakens security (modern OpenSSL salts automatically, but do not disable it).

What the exam asks. Mostly flag recall: "what does `-ciphers` do", "which mode name means Cipher Block Chaining" (`cbc`), "which flag decrypts" (`-d`), and reading a cipher name into algorithm/key size/mode.

5.2 dd (extracting the first bytes of a VeraCrypt container)

Purpose. Copy raw bytes from one file or device to another. In the VeraCrypt brute-force workflow (3.30) it pulls out the header for offline cracking.

Full syntax.

```
dd if=<input_file> of=<output_file> bs=<block_size> count=<n>
```

Breakdown of each part.

Part	Meaning
`dd`	The copy utility (data definition / disk dump)

Part	Meaning
`if=`	Input file or device
`of=`	Output file
`bs=`	Block size in bytes per read/write
`count=`	Number of blocks to copy

Flag table.

Flag	Meaning	When to use	Example
`if=`	Source path	Always	`if=target.tc`
`of=`	Destination path	Always	`of=target_hash.tc`
`bs=512`	Read/write 512 bytes at a time	VeraCrypt header extraction	`bs=512`
`count=1`	Copy exactly one block	Grabbing only the header	`count=1`
`status=progress`	Show progress (GNU dd)	Large copies	`status=progress`

Demo command (as shown on the slide, Windows `dd.exe`).

```
dd.exe if=E:\encrypted\target of=E:\encrypted\target_hash.tc \
    bs=512 count=1
```

Real sample output (captured in this sandbox with GNU dd on Linux, same command shape with a 4096-byte test file standing in for the container):

```
dd if=target of=target_hash.tc bs=512 count=1
```

```
1+0 records in
1+0 records out
512 bytes copied, 4.9453e-05 s, 10.4 MB/s
```

Reading it: "1+0 records in/out" means one full block was read and written, with zero partial blocks. "512 bytes copied" confirms exactly one 512-byte block was pulled out, matching `bs=512 count=1`.

Common mistakes. Using the wrong `bs` value changes how many bytes `count=1` actually copies (bs=512 count=1 is 512 bytes; bs=1 count=512 is also 512 bytes but much slower). Forgetting `count=` copies the whole source file. Mixing up `if=` and `of=` overwrites the wrong file.

What the exam asks. "Which command extracts raw bytes/the header from a container before cracking" -> `dd`. Reading `bs=` and `count=` to compute total bytes copied.

5.3 hashcat (brute-forcing the extracted VeraCrypt hash)

Purpose. Recover a password from a hash using mask (brute-force), dictionary, or hybrid attacks, with GPU acceleration.

Full syntax.

```
hashcat -a <attack_mode> -m <hash_mode> -w <workload> \
    <hashfile> <mask_or_wordlist>
```

Breakdown of each part.

Part	Meaning
`hashcat`	The cracking tool
`-a`	Attack mode (0 = wordlist, 3 = brute-force/ mask)
`-m`	Hash mode (which hash/container format to target)
`-w`	Workload profile (1 low to 4 insane GPU usage)
`<hashfile>`	File with the extracted hash (for example the `.tc` header)
`<mask_or_wordlist>`	A mask like `?d?d?d?d` or a wordlist file path

Flag table.

Flag	Meaning	When to use	Example
`-a 3`	Brute-force/mask attack	Unknown pattern, try all combos	`-a 3 ?d?d?d?d`
`-a 0`	Straight wordlist attack	Testing likely/default passwords	`-a 0 wordlist.txt`
`-m 13721`	VeraCrypt SHA-512 + XTS 512-bit (legacy mode, verify number for your hashcat version)	Target is a VeraCrypt container	`-m 13721`
`-w 1`	Low GPU workload	Keep the machine usable while cracking	`-w 1`
`?d`	Mask placeholder for one digit 0-9	Numeric-only passwords	`?d?d?d?d` = 4 digits
`?l ?u ?a`	Lowercase, uppercase, any printable (ADDED CONTEXT)	Building custom masks	`?u?l?l?l?d?d`

Demo commands (as shown on the slide).

```
hashcat.exe -a 3 -w 1 -m 13721 target_hash.tc ?d?d?d?d
hashcat.exe -w 1 -m 13721 target_hash.tc wordlist.txt
```

Sample output (illustrative; hashcat is not installed in this sandbox, so this is a representative output pattern, not a captured run).

```
hashcat (v6.2.6) starting

Hash.Mode........: 13721 (VeraCrypt SHA512 + XTS 512 bit)
Guess.Mask.......: ?d?d?d?d [4]
Speed.#1.........:   412.3 kH/s (0.15ms)
Recovered........: 1/1 (100.00%) Digests
Candidates.#1....: 7410 -> 9999

target_hash.tc:7421
```

Reading it: "Hash.Mode" confirms which `-m` was matched. "Speed" is guesses per second. "Recovered 1/1 (100%)" means the one hash was cracked. The last line `target_hash.tc:7421` is `<hashfile>:<password>`, so the recovered password is `7421`.

Common mistakes. Picking the wrong `-m` mode for the container version (VeraCrypt hash modes differ by hash algorithm and legacy vs current format; mismatched mode never cracks). Forgetting `-a 3` defaults to a straight attack that needs a wordlist, not a mask. Expecting a long alphanumeric password to crack quickly with a 4-digit mask (it will not, since the mask defines the search space).

What the exam asks. Which command cracks the extracted hash (`hashcat` or `John the Ripper`), what `-a 3` vs `-a 0` means, what a mask character like `?d` represents, and reading recovered-password output.

5.4 John the Ripper (alternative password cracker)

Purpose. Crack password hashes using wordlists, rules, or incremental (brute-force) modes; often used alongside or instead of hashcat.

Full syntax.

```
john [--format=<format>] [--wordlist=<file>] [--rules] <hashfile>
john --show <hashfile>
```

Breakdown of each part.

Part	Meaning
`john`	The cracking tool
`--format=`	Hash/container format to target
`--wordlist=`	Path to a dictionary file
`--rules`	Apply word-mangling rules (leet-speak, suffixes)

Part	Meaning
`--show`	Display already-cracked passwords from the pot file

Flag table.

Flag	Meaning	When to use	Example
`--wordlist=`	Dictionary attack	Testing known/ common passwords	`--wordlist=rockyou.txt`
`--rules`	Mutate wordlist entries	Users who tweak a base word	`--rules`
`--format=`	Select hash type/ container	Non-default hash formats	`--format=VeraCrypt`
`--show`	Print cracked results	After a session, review results	`john --show hash.txt`

Demo command in a realistic scenario.

```
john --format=VeraCrypt --wordlist=common-passwords.txt \
    target_hash.tc
john --show target_hash.tc
```

Sample output (illustrative pattern; John is not installed in this sandbox).

```
Loaded 1 password hash (VeraCrypt [PBKDF2-SHA512 256/256 AVX2])
Press 'q' or Ctrl-C to abort, almost any other key for status
summer2023       (target_hash.tc)
1g 0:00:00:42 DONE
```

Reading it: "Loaded 1 password hash" confirms the target was parsed. `summer2023` next to the filename is the cracked password. "1g" means one hash was guessed (cracked).

Common mistakes. Not specifying `--format=` when John cannot auto-detect a container type. Using a tiny or unrelated wordlist. Expecting John's default CPU speed to match hashcat's GPU speed on the same hardware.

What the exam asks. Choosing John vs hashcat for a scenario (both are acceptable "best tool" answers; the exam usually just wants "a password cracker" recognized), and reading cracked-password output.

5.5 md5sum / sha256sum (computing and checking digests)

Purpose. Compute or verify a file's hash locally without a web tool.

Full syntax.

```
md5sum [file...]
sha256sum [file...]
md5sum -c <checksum_file>
```

Breakdown of each part.

Part	Meaning
`md5sum` / `sha256sum`	Compute the named digest
`file...`	One or more files to hash (omit to read stdin)
`-c`	Check mode: verify files against a saved checksum list

Flag table.

Flag	Meaning	When to use	Example
(none)	Print `<hash> <filename>`	Generating a digest	`md5sum file.txt`
`-c`	Verify against a `.md5` / `.sha256` list	Confirming integrity later	`md5sum -c sums.md5`

Demo commands and real captured output (run in this sandbox).

```
printf 'hello' | md5sum
printf 'Hello' | md5sum
```

```
5d41402abc4b2a76b9719d911017c592  -
8b1a9953c4611296a827abf8c47804d7  -
```

```
md5sum secret.txt > sums.md5
md5sum -c sums.md5
echo "x" >> secret.txt
md5sum -c sums.md5
```

```
secret.txt: OK
secret.txt: FAILED
md5sum: WARNING: 1 computed checksum did NOT match
```

Reading it: the `-` after a hash means the input came from stdin, not a named file. "OK" means the file still matches the saved hash. After appending a byte, "FAILED" and the warning line show the file changed, exactly the avalanche effect and tamper-detection idea from 3.6.

Common mistakes. Comparing hashes by eye instead of with `-c` (easy to misread a long hex string). Hashing the wrong file or forgetting a file changed after the checksum was taken.

What the exam asks. Reading whether two digests match, what "OK" vs "FAILED" means, and which command (not a web tool) is appropriate when no internet access is allowed on a secure machine.

5.6 gpg (sign, encrypt, decrypt, verify)

Purpose. Implement OpenPGP operations: generate keys, encrypt and decrypt files, and create and verify signatures.

Full syntax.

```
gpg --full-generate-key
gpg --encrypt --recipient <email> <file>
gpg --decrypt <file>.gpg
gpg --detach-sign <file>
gpg --verify <file>.sig <file>
```

Breakdown of each part.

Part	Meaning
`--full-generate-key`	Create a new key pair
`--encrypt --recipient`	Encrypt a file to someone's public key
`--decrypt`	Decrypt a file using your private key
`--detach-sign`	Create a separate signature file
`--verify`	Check a signature against a file

Flag table.

Flag	Meaning	When to use	Example
`--armor`	Output text (ASCII) instead of binary	Pasting into email	`gpg --armor --encrypt ...`
`--recipient <email>`	Choose whose public key to use	Encrypting for someone	`--recipient bob@x.com`
`--list-keys`	Show keys in your keyring	Checking available public keys	`gpg --list-keys`
`--detach-sign`	Sign without bundling the file	Distributing file plus separate `.sig`	`gpg --detach-sign file`

Demo commands in a realistic scenario. A tester shares a findings report with a client using the client's public key.

```
gpg --list-keys
gpg --armor --encrypt --recipient client@example.com report.txt
gpg --decrypt report.txt.asc
```

Sample output (illustrative; this sandbox's gpg has no usable agent, so this is a representative pattern, not a captured run).

```
pub   rsa2048 2024-01-10 [E]
      1234 5678 90AB CDEF 1234 5678 90AB CDEF 1234 5678
uid           [ultimate] Client <client@example.com>

gpg: encrypted with 2048-bit RSA key, ID 90ABCDEF
```

Reading it: `pub rsa2048` shows the key type and size. The long hex string is the key **fingerprint**, which you should verify out of band (3.20, Web of Trust). "encrypted with ... key, ID 90ABCDEF" confirms which public key was used.

Common mistakes. Encrypting to the wrong recipient key, forgetting `--armor` when the output needs to go into a text email, and trusting a key without checking its fingerprint.

What the exam asks. Which command signs vs encrypts vs verifies, and the key-use rule from 3.15 (encrypt with the recipient's public key, sign with your own private key).

6. Ports, Protocols, Standards and Acronyms

6.1 Ports Used by Cryptographic Protocols

> **ADDED CONTEXT:** The notes do not list ports directly; these are standard, widely documented ports for the protocols this module covers (verify against current RFC/IANA assignments).

Port	Protocol	Simple purpose	Known weakness
443	HTTPS (HTTP over TLS/SSL)	Encrypted web traffic	Weak ciphers, old TLS, DROWN if SSLv2 shared
465	SMTPS (implicit TLS)	Encrypted mail submission	Same as above if misconfigured
587	SMTP with STARTTLS	Mail submission, upgrades to TLS	Downgrade if STARTTLS stripped
993	IMAPS	Encrypted mailbox access	Weak TLS config
995	POP3S	Encrypted mailbox access	Weak TLS config
636	LDAPS	Encrypted directory access	Weak TLS config

Port	Protocol	Simple purpose	Known weakness
22	SSH	Encrypted remote shell (uses its own key exchange, not SSL/TLS)	Weak host keys, old ciphers

6.2 Standards and Protocols

Standard/Protocol	Purpose	Related attack or tool
SSL (v2, v3)	Secure message transmission, Netscape	POODLE, DROWN
TLS (1.0-1.3)	Successor to SSL, handshake + record protocols	BEAST, CRIME, Sweet32
PGP / OpenPGP	Hybrid encryption, signing for files/email	GPG, Mailvelope
S/MIME	CA-based email signing/encryption	Outlook, SSL.com certs
X.509	Certificate format standard	CA, RA, VA (PKI)
PKCS #5	Password-based cryptography standard (defines PBKDF2)	Key stretching
FIPS 197	AES standard	AES
FIPS 186	Digital Signature Standard	DSA, ECDSA
FIPS 202	SHA-3 (Keccak) standard	SHA-3
FIPS 203/204/205	Post-quantum: ML-KEM, ML-DSA, SLH-DSA	Quantum-resistant migration
ANSI X9.31	Older RNG standard for financial crypto	DUHK attack
OpenPGP (RFC 4880/9580)	Open standard behind PGP/GPG	Web of Trust
TCG Opal	Self-encrypting drive standard	Hardware disk encryption
LUKS	Linux disk encryption format	Cryptsetup

Rare protocol introductions (ADDED CONTEXT):

- **OCSP (Online Certificate Status Protocol):** a live query ("is this certificate still valid?") that replaces downloading a whole CRL. Attackers care because a server can be configured to "soft-fail" (allow access if the OCSP check cannot be reached), which an attacker can force by blocking the OCSP request.
- **CRL (Certificate Revocation List):** a signed list of revoked certificate serial numbers published by a CA. Attackers care because stale or unchecked CRLs let a revoked (for example stolen) certificate keep working.

- **STARTTLS:** a command that upgrades a plaintext mail connection to TLS mid-session. Attackers care because a MITM can strip the STARTTLS command so the session stays plaintext (a STARTTLS-stripping downgrade attack).

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attacks and Their Countermeasures

Attack	Countermeasure
Brute-force (3.29)	Long keys/passphrases (128+ bits), key stretching, lockouts
Rainbow table (3.36)	Salt every hash, use slow KDFs (bcrypt, PBKDF2, Argon2)
Hash collision (3.33)	Retire MD5/SHA-1, use SHA-256 or SHA-3 for signatures
DUHK (3.34)	Patch firmware, drop ANSI X9.31, use NIST-approved DRBGs
DROWN (3.35)	Disable SSLv2 everywhere, do not reuse keys across protocols
Side-channel (3.32)	Constant-time code, masking, shielding, noise, limit physical access
Meet-in-the-middle (3.31)	Use one strong cipher with a long key, not stacked weak ciphers
Man-in-the-middle on key exchange (3.27)	Authenticate keys with certificates/signatures
Timing attack (3.27)	Constant-time cryptographic implementations, blinding
Rubber hose (3.27)	Policy, duress codes, split/shared secrets, legal protection
51% attack (3.37)	Large decentralized hash/stake power, more confirmations
Finney attack (3.38)	Wait for confirmations before delivering goods
Eclipse attack (3.39)	Diverse random peers, limit inbound connections, whitelist peers
Race attack (3.40)	Wait for confirmations, use well-connected nodes
DeFi sandwich attack (3.41)	Slippage limits, private transaction relays
Harvest-now-decrypt-later (3.42)	Migrate to post-quantum/hybrid key exchange now

Attack	Countermeasure
Weak RSA/short keys	Use RSA 2048+ bits or ECC, rotate and retire weak keys
Hard-coded keys in code	Secrets vaults, no keys in source/binaries, key derivation per use

7.2 Look-alike Comparisons

X vs Y	Difference in one line	How to tell them apart in a question
Symmetric vs Asymmetric	One shared key vs a public/private pair	"Same key both ways" = symmetric; "two keys" = asymmetric
Encryption vs Hashing	Reversible (with a key) vs one-way, no key	If it can be decrypted, it is encryption, not a hash
Digital signature vs MAC/HMAC	Asymmetric, gives nonrepudiation vs shared-secret, no nonrepudiation	"Cannot deny sending" = signature; "shared key check" = MAC
PKI vs Web of Trust	Hierarchy with CAs vs mesh where users vouch for each other	"CA issues" = PKI; "users sign each other's keys" = WoT
Signed certificate vs Self-signed certificate	Verified by a third-party VA vs verified by the user themself	"VA, CRL, OCSP" = CA-signed; "Java keytool/Keychain" = self-signed
SSL vs TLS	SSL is the older Netscape protocol; TLS is its secure successor	If the question names SSLv2/v3, it is legacy SSL; TLS 1.2/1.3 is current
PGP vs GPG	PGP is the original commercial-lineage product; GPG is the free OpenPGP tool	"Free, GNU" = GPG; generic "hybrid protocol" = PGP
TPM vs HSM	Built into the motherboard, one per machine vs external, shared device	"Chip on motherboard" = TPM; "external, manages many keys" = HSM
Linear vs Differential cryptanalysis	Known-plaintext, linear approximation vs chosen-plaintext, input/output differences	"Known plaintext" = linear; "chosen plaintext, differences" = differential
Ciphertext-only vs Known-plaintext vs Chosen-plaintext	Attacker has only ciphertext vs also some plaintext vs can pick the plaintext	Match what the attacker is said to control

X vs Y	Difference in one line	How to tell them apart in a question
Meet-in-the-middle vs Man-in-the-middle	Mathematical key-search technique vs active network interception	"Encrypt one end, decrypt the other, same plaintext/ciphertext known" = meet-in-the-middle; "intercepts live traffic" = MITM
Finney attack vs Race attack	Needs a pre-mined block vs no pre-mining, just fast broadcasting	"Pre-mined block" = Finney; "two transactions racing" = race
51% attack vs Eclipse attack	Controls majority of the whole network's power vs isolates one target node	"Majority of hash/stake power" = 51%; "isolates a node's peer table" = Eclipse
DROWN vs Heartbleed	Cross-protocol SSLv2 key-reuse attack vs OpenSSL memory-disclosure bug	"SSLv2 probes, same private key" = DROWN; "buffer over-read bug in OpenSSL" = Heartbleed (ADDED CONTEXT)
Brute force vs Rainbow table	Tries every key live vs looks up precomputed hashes	"Tries every combination" = brute force; "precomputed table" = rainbow table
Key stretching vs Salting	Slows down each guess (PBKDF2, bcrypt) vs makes each hash unique	"Adds iterations/cost" = stretching; "adds a random value before hashing" = salting

8. Exam Cheat Sheet

8.1 Keyword -> Answer Pairs

Keyword in the question	Likely answer
"Same key encrypts and decrypts"	Symmetric encryption
"Public key encrypts, private key decrypts"	Asymmetric encryption
"Letters replaced"	Substitution cipher
"Letters rearranged"	Transposition cipher
"Cannot be reversed, fixed output size"	Hash / message digest
"50 percent of output bits change"	Avalanche effect
"Chip on the motherboard"	TPM
"External device, manages many keys"	HSM
"Photons, polarization filters"	Quantum cryptography / QKD
"Issues and verifies certificates"	Certificate Authority (CA)
"Verifies the applicant for the CA"	Registration Authority (RA)

Keyword in the question	Likely answer
"Stores certificates, answers validity queries"	Validation Authority (VA)
"Verified via Java keytool/Keychain, no third party"	Self-signed certificate
"CRL, OCSP, CA chain"	CA-signed certificate
"Sign hash with sender private key"	Digital signature step
"Encrypt with recipient public key"	Confidentiality step (seal)
"Netscape, old protocol"	SSL
"Handshake Protocol + Record Protocol"	TLS
"Hybrid cryptosystem, random session key"	PGP
"Free OpenPGP implementation"	GPG
"Everyone is a CA, rings of trust"	Web of Trust
"On-the-fly encrypted volume"	VeraCrypt
"XTS-AES-128, 256-bit key, startup disk"	FileVault
"Distributed ledger of linked blocks"	Blockchain
"Known plaintext, linear approximation"	Linear cryptanalysis
"Chosen plaintext, input/output differences"	Differential cryptanalysis
"Holds b-k bits constant, SPN"	Integral cryptanalysis
"Quantum computer, Grover's, O(sqrt N)"	Quantum cryptanalysis
"Attacker has only ciphertext"	Ciphertext-only attack
"Torture or coercion for a key"	Rubber hose attack
"Measures exact execution time"	Timing attack
"Tries every possible key"	Brute-force attack
"dd then hashcat/John on a VeraCrypt file"	Brute-forcing VeraCrypt
"Encrypt one end, decrypt the other, same text"	Meet-in-the-middle attack
"Power, EM, light, timing, sound leakage"	Side-channel attack
"Two different inputs, same hash"	Hash collision attack
"Hard-coded seed, ANSI X9.31"	DUHK attack
"SSLv2 probes, same private key as TLS"	DROWN attack
"Precomputed hash table"	Rainbow table attack
"More than 50 percent of hash/stake power"	51% (majority) attack
"Pre-mined block, double spend"	Finney attack
"Isolates a node, fills peer table"	Eclipse attack
"Two transactions racing, no pre-mine"	Race attack
"Mempool, front-run, DEX"	DeFi sandwich attack

Keyword in the question	Likely answer
"Store encrypted data now, decrypt later with quantum"	Harvest-now-decrypt-later
"PKCS #5, applies HMAC with salt"	PBKDF2
"Blowfish-derived password hash"	bcrypt

8.2 Must-Memorize Numbers and Defaults

- Four objectives: Confidentiality, Integrity, Authentication, Nonrepudiation.
- DES key: 56 bits (64-bit field). 3DES: 112 or 168 bits. AES: 128/192/256 bits, 128-bit block.
- RSA minimum: 2048 bits. Symmetric minimum for large transactions: 256 bits. Hash minimum: 256 bits.
- MD5 digest: 128 bits (32 hex chars). SHA-1: 160 bits (40 hex chars). SHA-256: 256 bits (64 hex chars).
- SHA-1 rounds: 80. MD5 rounds: 64. MD4 rounds: 48. MD2 rounds: 18.
- ECC key range on the slide: 160 to 521 bits.
- Grover's algorithm: O(sqrt(N)) vs classical O(N); halves effective symmetric key strength.
- VeraCrypt hashcat header extraction: `bs=512 count=1`.
- Hashcat mask digit: `?d`. Attack mode 3 = brute-force/mask; mode 0 = wordlist.
- 5 blockchain creation steps: request, create, validate, add, share.
- 4 blockchain types: public, private, federated, hybrid.
- 13 named quantum-computing attacks (section 3.42).
- 12 general crypto-attack countermeasures; 10 quantum-attack countermeasures (sections 3.44 and 3.45).

8.3 "Pick the BEST Option" Rules for This Module

1. If two options both name a hash algorithm but one is MD5/SHA-1 and the other is SHA-256/SHA-3, the stronger modern hash is the BEST choice for "recommend a fix" questions.
2. If a question gives both "encrypt with recipient's public key" and "encrypt with sender's private key" as options for confidentiality, the recipient's public key is correct. Private-key signing options answer authentication/nonrepudiation questions, not confidentiality ones.
3. If a scenario names a specific tool used in the notes (VeraCrypt, GPG, hashcat, dd) alongside a generic alternative, prefer the named tool when the scenario matches the notes' use case.
4. For "what should you do first" after finding a weak algorithm (DES, RC4, MD5, SHA-1), the best next step is almost always "replace/retire it with the modern standard," not "monitor it" or "document it only."

5. For blockchain questions, match the distinguishing detail exactly: pre-mining = Finney, isolation of one node = Eclipse, majority power = 51%, no pre-mine race = Race, mempool front-running = sandwich.
6. For "best defense against quantum threats to symmetric keys," the answer is larger keys (for example AES-256), not "new algorithm," because Grover's attack only halves strength. For asymmetric keys (RSA, ECC), the best defense IS a new algorithm (post-quantum), since Shor's algorithm breaks them outright.

9. Practice Questions

Each question is followed immediately by its answer and explanation, as requested, instead of a separate answer key at the end.

Q1. A security analyst sees that a legacy payment terminal still uses an encryption algorithm with a 56-bit key and a 64-bit block. Which algorithm is this?

A. AES B. 3DES C. DES D. Blowfish

Answer: C. DES uses a 56-bit key (64-bit key field with parity) and a 64-bit block, exactly matching the scenario (section 3.4). A is wrong: AES uses 128/192/256-bit keys. B is wrong: 3DES uses 112 or 168-bit keys. D is wrong: Blowfish allows 32 to 448-bit variable keys.

Q2. Which of the following best describes asymmetric encryption?

A. The same key encrypts and decrypts the data B. A public key encrypts and a different private key decrypts C. Data is converted into a fixed-length digest that cannot be reversed D. The key is applied multiple times in nested rounds of hashing

A. Symmetric encryption, not asymmetric. **Answer: B.** This is the definition given on the slide (3.2): public key for encryption, private key for decryption. C describes a hash function (3.6), not encryption. D describes multi-layer hashing (3.9).

Q3. During a penetration test, you capture ciphertext encrypted with a reused one-time pad key across multiple messages. Which tool from the module's cryptanalysis toolset is designed for exactly this situation?

A. RsaCtfTool B. Msieve C. MTP D. CryptoSMT

A. RsaCtfTool targets weak RSA public keys, not OTP reuse. B. Msieve factors large integers, unrelated to OTP reuse. **Answer: C.** MTP (Many-Time Pad) is built specifically to attack a one-time pad key that was reused (section 3.43). D. CryptoSMT targets symmetric primitives via SMT/SAT solvers, a different cryptanalysis style.

Q4. A web server still allows SSLv2 connections using the same RSA private key as its modern TLS configuration. Which attack does this expose the server to?

A. DUHK B. DROWN C. Rainbow table D. Meet-in-the-middle

A. DUHK targets hard-coded RNG seeds, not shared private keys across protocols. **Answer: B.** DROWN specifically abuses SSLv2 support combined with a private key shared with TLS to decrypt TLS sessions (section 3.35). C. Rainbow tables target password hashes, unrelated to TLS/SSLv2. D. Meet-in-the-middle is a mathematical key-search technique against double encryption, not a protocol-sharing flaw.

Q5. Which hashcat command flag tells it to try every possible value for a mask pattern, such as `?d?d?d?d`, rather than reading from a wordlist?

A. `-a 0` B. `-a 3` C. `-m 0` D. `--show`

A. `-a 0` is the straight wordlist attack mode. **Answer: B.** `-a 3` selects brute-force/mask attack mode, used with a mask like `?d?d?d?d` (section 5.3). C. `-m` selects the hash mode/type, not the attack mode. D. `--show` is a John the Ripper flag to display already-cracked results, not a hashcat attack-mode flag.

Q6. What is the correct command sequence to brute-force a VeraCrypt container's password offline?

A. Run hashcat directly on the .hc container file B. Use `dd` to extract the first 512 bytes, then run hashcat or John against that extracted hash file C. Use `openssl enc -d` to decrypt the container directly D. Use CyberChef to compute a rainbow table of the container

A. hashcat needs the extracted header, not the whole container file. **Answer: B.** The notes show exactly this two-step workflow: `dd if=<container> of=<hashfile> bs=512 count=1`, then `hashcat -m 13721 <hashfile> <mask or wordlist>` (section 3.30 and 5.2/5.3). C. OpenSSL `enc -d` needs the correct cipher and key already known; it cannot blind-decrypt a VeraCrypt container. D. CyberChef is not used to build rainbow tables for VeraCrypt in the notes; this is not the documented workflow.

Q7. A company's PKI issues a certificate after a Registration Authority confirms the applicant's identity. Later, a client wants to know if that certificate is still valid. Which PKI component answers that query?

A. Certificate Authority (CA) B. Registration Authority (RA) C. Validation Authority (VA) D. End user

A. The CA issues certificates but the ongoing validity check is handled elsewhere per the slide's flow. B. The RA only verifies applicants before issuance, not validity later. **Answer: C.** The VA stores certificates and their public keys and answers validity queries, per the module's PKI diagram (section 3.13). D. The end user requests and uses certificates, but does not answer validity queries for others.

Q8. Which statement correctly completes the digital signature process: the sender signs the hash of a message with ___, and the receiver verifies it with ___.

A. the sender's public key; the sender's private key B. the sender's private key; the sender's public key C. the receiver's public key; the receiver's private key D. the receiver's private key; the sender's public key

A. This reverses the correct order. **Answer: B.** Sign with the sender's own private key; anyone verifies with the matching sender's public key (section 3.15). C and D mix in the receiver's keys, which are used for confidentiality (sealing/opening), not for signing/verifying.

Q9. An attacker finds two different files that produce the exact same SHA-1 hash value, then attaches a victim's valid signature (made over one file) to the other file. What is this attack called?

A. Rainbow table attack B. Hash collision attack C. Side-channel attack D. DUHK attack

A. Rainbow tables reverse hashes using precomputed tables; they do not rely on two inputs sharing one hash. **Answer: B.** This is the exact scenario described for hash collision attacks: forging a signature from message a1 onto message a2 because both share a hash (section 3.33). C. Side-channel attacks exploit physical leakage (power, timing), not hash collisions. D. DUHK targets predictable random number generators, unrelated to hash collisions.

Q10. A blockchain attacker surrounds a single target node with malicious nodes by poisoning its peer table, cutting it off from legitimate nodes. What is this attack called?

A. 51% attack B. Finney attack C. Eclipse attack D. Race attack

A. A 51% attack needs majority control of the whole network's power, not isolation of one node. B. A Finney attack relies on a pre-mined block, not peer-table poisoning. **Answer: C.** This is the Eclipse attack: filling a node's peer table with malicious IPs so it only connects to the attacker (section 3.39). D. A Race attack relies on broadcasting two conflicting transactions quickly, not isolating a node.

Q11. Which of the following is the BEST long-term defense against an attacker who is secretly recording today's encrypted TLS traffic, planning to decrypt it once a powerful quantum computer exists?

A. Rotate the TLS certificate every 90 days B. Switch to a longer symmetric key such as AES-256 only C. Migrate key exchange to post-quantum or hybrid classical/post-quantum algorithms D. Disable SSLv2 on all servers

A. Certificate rotation does not protect data already recorded, since the key exchange algorithm itself is still breakable by a quantum computer. B. AES-256 helps against Grover's algorithm on the symmetric cipher, but the bigger risk here is the asymmetric key exchange (RSA/ECC/DH), which Shor's algorithm breaks outright. **Answer: C.** This is the harvest-now-decrypt-later threat (section 3.42); the real fix is moving the key exchange itself to post-quantum or hybrid algorithms (section 3.45). D. Disabling SSLv2 defends against DROWN, a different and unrelated attack.

Q12. Which statement about bcrypt and PBKDF2 is correct?

A. PBKDF2 is based on a derivation of the Blowfish algorithm B. bcrypt is defined in PKCS #5 v2.01 C. PBKDF2 applies a function such as HMAC to the password and a salt to produce a derived key D. Both algorithms are fast by design, to support high-volume logins

A. This describes bcrypt, not PBKDF2. B. PKCS #5 v2.01 defines PBKDF2, not bcrypt. **Answer: C.** This matches the slide's description of PBKDF2 exactly (section 3.46). D. Both are deliberately SLOW (key stretching), not fast, to resist brute-force guessing.

Q13. A tester wants to verify that a downloaded Linux ISO was not tampered with. The vendor publishes a SHA-256 value on their site. What should the tester do?

A. Open the ISO in a hex editor and visually compare bytes B. Compute the SHA-256 of the downloaded file and compare it to the published value C. Encrypt the ISO with AES-256 and check the file size D. Use a rainbow table to look up the ISO's hash

A. Visual byte comparison of a multi-gigabyte file is impractical and not how integrity checking works. **Answer: B.** This is the standard use of a message digest for integrity checking (section 3.6): recompute and compare. C. Encrypting the file does not check integrity, and file size does not confirm content tampering. D. Rainbow tables reverse weak password hashes; they are not used to verify file integrity.

Q14. Which type of cipher shifts the positions of plaintext letters without changing the letters themselves?

A. Substitution cipher B. Transposition cipher C. Block cipher D. Stream cipher

A. Substitution replaces letters/symbols; positions do not move. **Answer: B.** Transposition rearranges letter positions while keeping the letters unchanged (section 3.3). C and D describe modern ciphers grouped by how they process input data (fixed blocks vs continuous stream), not classical letter-shuffling.

Q15. An organization's email client is configured so that outgoing mail is signed with a certificate from a trusted CA and the message is also encrypted. Which email security standard is being used?

A. PGP B. GPG C. S/MIME D. Web of Trust

A and B. PGP/GPG use the Web of Trust model, not CA-issued certificates in Outlook's Trust Center. **Answer: C.** The Outlook configuration shown in the notes (Trust Center -> Email Security -> S/MIME certificate) is S/MIME (section 3.21). D. Web of Trust is the PGP/GnuPG trust model, unrelated to Outlook's CA-based S/MIME setup.

Q16. A security team measures slight differences in the time an embedded device takes to perform RSA decryption, hoping to recover bits of the private key. What category of attack is this?

A. Brute-force attack B. Side-channel attack (timing attack) C. Hash collision attack D. Meet-in-the-middle attack

A. Brute force tries every key; it does not use physical measurements. **Answer: B.** This matches the timing attack described under both the cryptography attacks list and side-channel attacks (sections 3.27 and 3.32): measuring exact execution times of modular exponentiation. C. Hash collisions involve finding two inputs with the same hash, not timing measurements. D. Meet-in-the-middle is a key-search technique for double encryption, unrelated to timing measurements.

Q17. Which blockchain attack does NOT require the attacker to pre-mine a block, relying instead purely on network broadcast speed?

A. Finney attack B. Race attack C. 51% attack D. Eclipse attack

A. The Finney attack specifically requires a pre-mined block held back until the victim's payment is accepted (section 3.38). **Answer: B.** The Race attack explicitly needs no pre-mining; it wins by broadcasting a conflicting transaction faster than the network confirms the first one (section 3.40). C. A 51% attack needs sustained majority computing/staking power, not a single pre-mined block. D. An Eclipse attack isolates a node via peer-table poisoning, unrelated to pre-mining or broadcast speed.

Q18. You are asked to recommend an immediate fix for a web application storing user passwords as unsalted MD5 hashes. What is the BEST recommendation?

A. Switch to unsalted SHA-256 hashes B. Switch to a salted, slow KDF such as bcrypt or PBKDF2 C. Encrypt the MD5 hashes with AES-256 before storing them D. Add a CAPTCHA to the login page

A. Still unsalted, so still vulnerable to rainbow tables, just with a different algorithm. **Answer: B.** Salting defeats precomputed rainbow tables, and a slow KDF (key stretching) makes brute-forcing the leaked hashes far more costly (sections 3.36 and 3.46). This is the best combined fix. C. Encrypting a hash does not address the core weakness; the encryption key itself becomes a new single point of failure. D. CAPTCHA may slow online guessing attempts but does nothing to protect already-leaked hash data.

Q19. In the CEH v13 AI context, how can an AI assistant help a defender during certificate-based threat monitoring?

A. By automatically revoking any certificate it finds suspicious B. By scanning Certificate Transparency logs for look-alike domains and flagging them for review C. By generating new root CA private keys on demand D. By bypassing OCSP checks to speed up page loads

A. An AI assistant should flag for human review, not autonomously revoke certificates; that authority belongs to the CA/organization. **Answer: B.** This matches the AI angle described for PKI (section 3.13): monitoring Certificate Transparency logs for domains

that mimic a legitimate brand. C. Generating CA root keys is a sensitive operation requiring strict controls, not something delegated to an AI assistant casually. D. Bypassing OCSP checks weakens security and is the opposite of a defensive AI use case.

Q20. Which flag, when used with `openssl enc`, ensures the encryption key is derived from the password using a strong, modern key derivation function instead of a weaker legacy method?

A. `-salt` B. `-base64` C. `-pbkdf2` D. `-d`

A. `-salt` adds randomness to the derivation, which is good practice, but does not by itself select the stronger derivation algorithm. B. `-base64` only changes the output encoding to text. **Answer: C.** `-pbkdf2` tells OpenSSL to use PBKDF2 for deriving the key from the password (section 5.1), which is the recommended, modern option. D. `-d` means decrypt; it does not affect key derivation strength.

Q21. A tester finds a device whose VPN uses the ANSI X9.31 random number generator with a seed value hard-coded into the firmware. Which named attack applies directly to this weakness?

A. DROWN B. DUHK C. Eclipse attack D. Rainbow table attack

A. DROWN concerns SSLv2/TLS private-key reuse, unrelated to RNG seeds. **Answer: B.** DUHK (Don't Use Hard-Coded Keys) directly targets devices using ANSI X9.31 with a hard-coded seed key (section 3.34). C. Eclipse attacks target blockchain peer-to-peer networking, unrelated to RNG weaknesses. D. Rainbow tables reverse password hashes, unrelated to RNG seed compromise.

Q22. What is the main practical difference between a Hardware Security Module (HSM) and a Trusted Platform Module (TPM)?

A. HSM is built into the motherboard; TPM is an external shared device B. TPM is built into the motherboard of one machine; HSM is typically an external device serving many systems C. They are different names for the exact same hardware D. TPM only works with asymmetric algorithms; HSM only works with symmetric algorithms

A. This reverses the correct definitions. **Answer: B.** Per the notes (section 3.10), TPM is a chip on the motherboard of a single machine, while HSM is an additional external device used for managing and storing keys, often shared across systems. C. They serve related but distinct roles and are not interchangeable terms. D. Neither device is restricted to only one algorithm family.

Q23. Which attack type gives the attacker the ability to choose their own ciphertexts and obtain the corresponding plaintexts?

A. Known-plaintext attack B. Chosen-plaintext attack C. Chosen-ciphertext attack D. Ciphertext-only attack

A. Known-plaintext means the attacker already possesses some plaintext and matching ciphertext, not that they chose it. B. Chosen-plaintext means the attacker picks the plaintext and observes the resulting ciphertext, the reverse direction. **Answer: C.** Chosen-ciphertext attacks let the attacker pick arbitrary ciphertexts and obtain their decrypted plaintexts (section 3.27). D. Ciphertext-only gives the attacker no ability to choose or supply anything; they only passively observe ciphertext.

Q24. A smart contract trader notices a large pending swap in the mempool, quickly places a buy order ahead of it, lets the victim's trade push the price up, then sells for profit. What is this called?

A. 51% attack B. DeFi sandwich attack C. Finney attack D. Meet-in-the-middle attack

A. A 51% attack needs majority network power, not mempool monitoring of a single trade. **Answer: B.** This is the DeFi sandwich attack exactly as described in section 3.41: watching the mempool, front-running, and profiting from the price change. C. A Finney attack involves a pre-mined block and double-spending, not front-running a DEX trade. D. Meet-in-the-middle is a cryptographic key-search technique, unrelated to DeFi trading.

Q25. Why is Triple DES (3DES) considered to provide only about 112-bit security even though its key length can be 168 bits?

A. Because 3DES actually only encrypts with a 56-bit key internally B. Because of the meet-in-the-middle attack on multiple-encryption schemes, which reduces the effective work factor C. Because 3DES uses a weak S-box compared to AES D. Because 3DES was never standardized

A. 3DES does use 56-bit DES keys internally (up to three of them), but that alone does not explain the "112-bit" effective-security figure. **Answer: B.** The meet-in-the-middle attack (section 3.31) on stacked encryption reduces the effective strength of multiple-key schemes like 3DES, which is why its practical security (about 112 bits) is lower than its raw key length (168 bits) would suggest. C. 3DES reuses the DES S-boxes; the weakness discussed in the notes is about key-stacking math, not S-box design. D. 3DES is a standardized, if legacy, algorithm (used in financial systems per section 3.4).

Q26. A penetration tester wants an AI assistant to help brute-force a weak VeraCrypt container password, using facts they legitimately gathered about the container's owner during an authorized engagement. What is the MOST appropriate way to use the AI assistant here?

A. Ask it to directly connect to the target and crack the password itself B. Ask it to generate candidate password patterns and matching hashcat mask or rule strings from the gathered facts, which the tester then runs locally C. Ask it to guess the password outright without any tool D. Ask it to bypass VeraCrypt's encryption entirely

A. AI assistants do not have the ability to connect to and attack systems directly; the human tester runs the actual tools. **Answer: B.** This matches the AI angle described in section 3.30: using AI to turn known facts into realistic password patterns and the corresponding hashcat mask/rule syntax, which the tester then executes within the authorized engagement. C. An AI assistant cannot reliably "guess" a specific password outright; it can only suggest likely patterns to test. D. There is no way to "bypass" strong encryption; the only realistic path is guessing a weak password faster, not breaking the math.

10. Flashcards

Q: What are the four objectives of cryptography? | A: Confidentiality, Integrity, Authentication, Nonrepudiation. Q: What key does symmetric encryption use for encrypt and decrypt? | A: The same single shared key. Q: What two keys does asymmetric encryption use? | A: A public key and a mathematically linked private key. Q: What does a substitution cipher do? | A: Replaces plaintext units with other symbols. Q: What does a transposition cipher do? | A: Rearranges the positions of plaintext letters. Q: What is a block cipher? | A: A cipher that encrypts fixed-size chunks of data. Q: What is a stream cipher? | A: A cipher that encrypts a continuous stream of data. Q: What key size and block size does DES use? | A: 56-bit key, 64-bit block. Q: What key sizes does AES support? | A: 128, 192, and 256 bits, with a 128-bit block. Q: Name two AES competition finalists other than Rijndael. | A: Serpent and Twofish (also RC6 and MARS). Q: What algorithm provides signatures only, not encryption? | A: DSA (Digital Signature Algorithm). Q: What problem does Diffie-Hellman solve? | A: Secure key exchange over an insecure channel. Q: What is the avalanche effect? | A: Changing one input bit gives each output bit a 50 percent chance of flipping. Q: Can a message digest be reversed? | A: No, it is a one-way hash function. Q: How many bits does an MD5 digest have? | A: 128 bits. Q: How many bits does a SHA-1 digest have, and how many rounds? | A: 160 bits, 80 rounds. Q: What internal design does SHA-3 use? | A: The sponge construction. Q: What is multi-layer (nested) hashing? | A: Applying a hash function repeatedly to the input or prior output. Q: What tool can chain hashing and encoding operations as a "recipe"? | A: CyberChef. Q: What is a TPM? | A: A crypto-processor chip on the motherboard that stores keys and does crypto operations. Q: What is an HSM? | A: An external device for managing, generating, and securely storing cryptographic keys. Q: What does quantum key distribution (QKD) use to carry data? | A: Photons with polarization, sent through filters. Q: What are the six PKI components named in the notes? | A: Certificate management system, digital certificates, VA, CA, end user, RA. Q: Who verifies the applicant before a CA issues a certificate? | A: The Registration Authority (RA). Q: Who answers whether a certificate is still valid? | A: The Validation Authority (VA), using CRL or OCSP. Q: What tools can create a self-signed certificate? | A: Adobe Acrobat Reader, Java keytool, Apple Keychain. Q: With what key does a sender sign a digital signature? | A: Their own private key. Q: With what key does a sender encrypt a message for confidentiality? | A: The recipient's public key. Q: What does SSL stand for, and who

developed it? | A: Secure Sockets Layer, developed by Netscape. Q: What two sub-protocols make up TLS? | A: The Handshake Protocol and the Record Protocol. Q: In the SSL/TLS handshake, whose public key encrypts the premaster secret? | A: The server's public key. Q: Why is PGP called a hybrid cryptosystem? | A: It combines symmetric encryption of data with asymmetric encryption of the session key. Q: What open standard does GPG implement for free? | A: OpenPGP. Q: What trust model do PGP, OpenPGP, and GnuPG use? | A: The Web of Trust. Q: In Outlook, where do you set S/MIME signing and encryption certificates? | A: File > Options > Trust Center > Trust Center Settings > Email Security > Settings. Q: What encryption technology and key size does FileVault use? | A: XTS-AES-128 with a 256-bit key. Q: What is a hidden volume in disk encryption? | A: A second encrypted volume hidden inside the free space of another one. Q: What are the 5 steps to create a blockchain? | A: Request, create a block, validate, add to the chain, share the ledger. Q: Name the 4 types of blockchain from the notes. | A: Public, private, federated, hybrid. Q: Linear cryptanalysis is what kind of attack? | A: A known-plaintext attack using a linear approximation of the cipher. Q: Differential cryptanalysis originally needed what kind of plaintext access? | A: Chosen plaintext. Q: What does Grover's algorithm reduce search time to? | A: From O(N) to O(square root of N). Q: What attack uses only ciphertext to try to recover the key? | A: Ciphertext-only attack. Q: What attack extracts a key through coercion or torture? | A: Rubber hose attack. Q: What two commands does the notes' VeraCrypt brute-force workflow use? | A: dd to extract the header, then hashcat or John the Ripper to crack it. Q: What hashcat flag selects brute-force (mask) attack mode? | A: -a 3. Q: What does the meet-in-the-middle attack exploit? | A: Multiple encryption schemes, by encrypting from one end and decrypting from the other to meet in the middle. Q: Name four physical channels a side-channel attack can monitor. | A: Power consumption, electromagnetic field, light emission, and timing/sound. Q: What does a hash collision attack let an attacker do? | A: Forge a signature from one message onto a different message with the same hash. Q: What does DUHK stand for and what RNG does it target? | A: Don't Use Hard-Coded Keys; targets the ANSI X9.31 random number generator. Q: What makes a DROWN attack possible? | A: A server supporting SSLv2 and TLS with the same private key. Q: What does a rainbow table attack use to crack hashes quickly? | A: A precomputed table of hash values for word lists. Q: What defeats rainbow table attacks? | A: Salting each hash before storing it. Q: What is a 51% (majority) attack? | A: Controlling more than half of a blockchain's hash rate or staking power. Q: What makes a Finney attack different from a Race attack? | A: Finney needs a pre-mined block; Race does not. Q: What does an Eclipse attack do to a blockchain node? | A: Isolates it by filling its peer table with malicious node addresses. Q: What does a DeFi sandwich attack exploit? | A: Mempool visibility and transaction ordering on a DEX to front-run a trade. Q: What is the harvest-now-decrypt-later threat? | A: Storing encrypted data today to decrypt it later once quantum computers can break the keys. Q: What is key stretching? | A: Strengthening a weak key by making the derivation process longer or slower. Q: What standard defines PBKDF2? | A: PKCS #5 version 2.01. Q: What algorithm is bcrypt based on? | A: A derivation of the Blowfish algorithm. Q: What is the minimum

recommended RSA key size for secure applications? | A: 2048 bits. Q: What is the minimum recommended hash length for secure applications? | A: 256 bits.

11. Glossary

Adaptive chosen-plaintext attack - An attack where the attacker sends interactive queries, choosing each plaintext based on earlier results.

Advanced Encryption Standard (AES) - A symmetric block cipher with 128, 192, or 256-bit keys and a 128-bit block, the current government and industry standard.

Asymmetric encryption - Encryption using a public key to encrypt and a different, linked private key to decrypt.

Avalanche effect - A property where changing one input bit gives roughly half the output bits a chance to flip.

bcrypt - A password-hashing function derived from Blowfish, used for key stretching.

Birthday attack - A method of finding a hash collision in about the square root of the total possible hash values.

Block cipher - A cipher that encrypts data in fixed-size chunks.

Blockchain - A distributed ledger of linked, hash-chained blocks of transactions.

Brute-force attack - Trying every possible key until the correct one is found.

Certificate Authority (CA) - The PKI entity that issues and verifies digital certificates.

Certificate Revocation List (CRL) - A signed list of certificates a CA has revoked before their expiry.

Chosen-ciphertext attack - An attack where the attacker picks ciphertexts and obtains their decrypted plaintexts.

Chosen-plaintext attack - An attack where the attacker picks plaintexts and observes the resulting ciphertexts.

Cipher - An algorithm used to encrypt or decrypt data.

Ciphertext - The scrambled output of encryption.

Ciphertext-only attack - An attack using only captured ciphertext, with the goal of recovering the key.

CyberChef - A free web tool (from GCHQ) for chaining encoding, hashing, and encryption operations into a "recipe."

Diffie-Hellman - An asymmetric algorithm used to exchange a shared secret key over an insecure channel.

Differential cryptanalysis - Analyzing how differences in plaintext inputs affect differences in ciphertext outputs.

Digital certificate - An electronic document binding an identity to a public key, signed by a CA.

Digital Signature Algorithm (DSA) - An asymmetric algorithm used only for creating digital signatures.

Digital signature - A cryptographic value, created with a private key, that proves authenticity, integrity, and nonrepudiation of a message.

Disk encryption - Converting all or part of a disk's data into unreadable form, protecting it even when the OS is offline.

DROWN attack - A cross-protocol attack that decrypts TLS traffic by abusing a server's SSLv2 support with the same private key.

DUHK attack - An attack that recovers encryption keys from devices using the ANSI X9.31 RNG with a hard-coded seed.

Eclipse attack - A blockchain attack that isolates a target node by surrounding it with malicious peers.

ElGamal - An asymmetric algorithm used for encryption and key exchange.

Elliptic Curve Cryptography (ECC) - An asymmetric algorithm family offering strong security with small key sizes.

Finney attack - A double-spend blockchain attack using a pre-mined block held back until a victim accepts payment.

51% (majority) attack - Controlling over half of a blockchain's mining or staking power to manipulate the chain.

Frequency analysis - Studying how often letters or letter groups appear in ciphertext to break simple substitution ciphers.

Grover's algorithm - A quantum algorithm that searches an unsorted space in O(square root of N) instead of O(N), halving effective symmetric key strength.

Hardware Security Module (HSM) - An external device dedicated to generating, managing, and storing cryptographic keys.

Harvest-now-decrypt-later - Storing encrypted data today with the plan to decrypt it later using a future quantum computer.

Hash collision attack - Finding two different inputs that produce the same hash, enabling signature forgery.

Hash function / message digest - A one-way function that produces a fixed-size digest from any input.

Integral cryptanalysis - A cryptanalysis method that holds some bits constant and varies others to find patterns in substitution-permutation networks.

Key stretching - Strengthening a weak key by making its derivation process longer or slower (for example PBKDF2, bcrypt).

Known-plaintext attack - An attack where the attacker knows part of the plaintext and its matching ciphertext.

Linear cryptanalysis - A known-plaintext attack that uses a linear approximation of a block cipher's behavior.

Man-in-the-middle attack - An attack where the adversary intercepts and potentially alters communication between two parties, often during key exchange.

Meet-in-the-middle attack - A key-search technique against multiple encryption that encrypts from one end and decrypts from the other.

Message Digest (MD) family - A series of hash algorithms (MD2, MD4, MD5, MD6) producing 128-bit (or variable, for MD6) digests.

Nonrepudiation - A property ensuring the sender cannot deny having sent a message.

One-time pad (OTP) - An encryption method using a random key as long as the message, used only once.

OpenPGP - The open standard behind PGP and GnuPG (RFC 4880/9580).

Plaintext - Readable, unencrypted data.

Post-quantum cryptography (PQC) - Cryptographic algorithms designed to resist attacks from quantum computers.

Pretty Good Privacy (PGP) - A hybrid cryptosystem protocol for encrypting, decrypting, and signing data, files, and email.

Public Key Infrastructure (PKI) - The hardware, software, people, policies, and procedures for managing digital certificates.

Quantum cryptanalysis - Using a quantum computer to break cryptographic algorithms.

Quantum key distribution (QKD) - A method of distributing encryption keys using quantum mechanics (such as photon polarization).

Race attack - A double-spend blockchain attack using two conflicting transactions broadcast in quick succession, without pre-mining.

Rainbow table attack - Reversing hash values using a precomputed table of hashes for likely passwords.

Registration Authority (RA) - The PKI entity that verifies an applicant's identity before a CA issues a certificate.

RSA - An asymmetric algorithm (Rivest-Shamir-Adleman) based on the difficulty of factoring large numbers, used for encryption, signatures, and key exchange.

Rubber hose attack - Extracting a cryptographic secret from a person through coercion or torture.

S/MIME - Secure/Multipurpose Internet Mail Extensions, a CA-based standard for signing and encrypting email.

Salt - A random value added to a password before hashing, to defeat precomputed hash tables.

Secure Sockets Layer (SSL) - An older Netscape-developed protocol for securing data transmitted over the Internet, succeeded by TLS.

Side-channel attack - A physical attack that extracts secrets by monitoring a device's power, timing, electromagnetic, or acoustic emissions.

Stream cipher - A cipher that encrypts a continuous stream of data, often bit by bit.

Substitution cipher - A classical cipher that replaces plaintext units with other symbols.

Symmetric encryption - Encryption that uses the same key for both encryption and decryption.

Timing attack - A side-channel attack that measures exact execution times to infer key information.

Transport Layer Security (TLS) - The modern successor to SSL, consisting of a Handshake Protocol and a Record Protocol.

Transposition cipher - A classical cipher that rearranges the positions of plaintext letters.

Trusted Platform Module (TPM) - A crypto-processor chip built into a computer's motherboard that securely stores keys.

Validation Authority (VA) - The PKI entity that stores certificates and answers queries about certificate validity.

VeraCrypt - A free tool for creating and maintaining on-the-fly encrypted disk volumes.

Web of Trust (WoT) - A decentralized trust model (used by PGP, OpenPGP, and GnuPG) where users sign each other's keys.

X.509 - The standard format for digital certificates.

XTS mode - A block cipher mode designed for disk encryption, used by FileVault and BitLocker.

12. Coverage Report

12.1 Page-to-Section Mapping

Page(s)	Topic	Guide section(s)
1	Objectives, cryptography, symmetric/asymmetric	1, 3.1, 3.2
2	Ciphers, symmetric algorithms	3.3, 3.4
3	Asymmetric algorithms, message digest	3.5, 3.6
4	Digest function comparison, calculators	3.7, 3.8
5	Multi-layer hashing, hardware encryption	3.9, 3.10
6	Quantum cryptography, cryptography tools	3.11, 3.12
7	PKI, signed vs self-signed certificates	3.13, 3.14
8	Digital signature, SSL	3.15, 3.16
9	TLS, PGP	3.17, 3.18
10	GPG, Web of Trust	3.19, 3.20
11	S/MIME in Outlook, email tools	3.21, 3.22
12	Disk encryption, disk encryption tools	3.23, 3.24
13	Blockchain, cryptanalysis methods (intro)	3.25, 3.26

Page(s)	Topic	Guide section(s)
14-15	Cryptanalysis formulas (linear, differential, integral, quantum), attack types	3.26, 3.27
16	Attack types (cont'd), code breaking methodologies	3.27, 3.28
17	Brute-force attack, brute-forcing VeraCrypt	3.29, 3.30, 5.2, 5.3
18	Meet-in-the-middle, side-channel	3.31, 3.32
19	Hash collision, DUHK	3.33, 3.34
20	DROWN, rainbow table	3.35, 3.36
21	51% attack, Finney attack	3.37, 3.38
22	Eclipse attack, Race attack	3.39, 3.40
23	DeFi sandwich attack, quantum computing attacks	3.41, 3.42
24	Cryptanalysis tools, defend against crypto attacks	3.43, 3.44
25	Defend against quantum attacks, key stretching	3.45, 3.46
26	Module summary	1 (summary table)

12.2 [UNCLEAR] Items

- Page 5 (slide 11): exact order of CyberChef recipe operations and the HMAC key value were small in the screenshot (section 3.9).
- Page 6 (slide 14): BCTextEncoder byte counts ("282 B", "622 B") and which link belongs to BCTextEncoder vs the other listed tools (section 3.12).
- Page 9 (slide 20): the Record Protocol's example cipher "DES" is stated by the slide but is cryptographically obsolete for real TLS deployments (section 3.17).
- Page 10 (slide 23): the exact connecting lines of the Web of Trust diagram between Alice, Alen, Bob, Smith, Henry, and John were hard to follow in the screenshot (section 3.20).
- Page 11 (slide 25): most of the RMail/RPost eSignOff screenshot text (names, IP address) was below reliable legibility (section 3.22).
- Page 17 (slide 35): the John the Ripper screenshot's version string "0.4beta4" does not match John's standard versioning; best reading only (section 3.30).
- Page 18 (slide 36): the meet-in-the-middle diagram's exact labels for the 256-key toy example were small in the screenshot (section 3.31).
- Page 24 (slide 50): the Enigma ciphertext string shown in the CrypTool screenshot was transcribed on a best-effort basis (section 3.43).

12.3 Items to Double-Check (flagged "verify" in the text)

- Exact NIST SP 800-57 strength-equivalence table numbers (section 3.5).
- The precise MAXDEPTH definition and current post-quantum cryptanalysis resource parameters (section 3.26).
- Current hashcat VeraCrypt hash-mode numbers, since mode 13721 is marked "legacy" in newer hashcat releases, which use different mode numbers for current VeraCrypt headers (sections 3.30 and 5.3).
- Current NIST PQC standard numbers and status: FIPS 203/204/205 are finalized (ML-KEM, ML-DSA, SLH-DSA, August 2024); FIPS 206 (FN-DSA) and HQC status may have changed since this guide was written (section 3.45).
- The DROWN CVE identifier and exact year (section 3.35).
- Current OWASP-recommended PBKDF2 iteration counts (section 3.46).

12.4 Confirmation of Coverage

Every line in the page-by-page inventory (work file, not delivered) was placed into at least one section of this guide: learning objectives (section 1), cryptography concepts and types (3.1-3.2), ciphers and all 17 symmetric plus 5 asymmetric algorithms (3.3-3.5), all 13 hash algorithms and their calculators (3.6-3.9), hardware encryption and quantum cryptography (3.10-3.11), the cryptography tools list (3.12), PKI and certificates (3.13-3.14), digital signatures and SSL/TLS (3.15-3.17), PGP, GPG, Web of Trust, S/MIME, and email tools (3.18-3.22), disk encryption and its tools (3.23-3.24), blockchain (3.25), all four cryptanalysis methods with their formulas (3.26), all 11 named cryptography attacks (3.27), code-breaking methodologies (3.28), brute force and the VeraCrypt demo with its exact commands (3.29-3.30), meet-in-the-middle and side-channel attacks (3.31-3.32), hash collision, DUHK, DROWN, and rainbow table attacks (3.33-3.36), all five blockchain attacks (3.37-3.41), all 13 quantum computing attacks (3.42), cryptanalysis tools (3.43), both countermeasure lists (3.44-3.45), and key stretching (3.46). Tool and command details are additionally cross-referenced in sections 4 and 5. No inventory line was left unplaced.